



Instituto Nuevos Aires

Diseño y Programación Web

Sistema de monitoreo personal

Prácticas Profesionalizantes presentadas para obtener el
Título de Diseño y Programación Web

Estudiante: Diego Comisso

Tutores: Johanna Yamila Tabella

Año 2026

Resumen

En la actualidad, el avance de las tecnologías electrónicas y de las comunicaciones ha permitido el desarrollo de dispositivos capaces de monitorear en tiempo real distintos parámetros relacionados con la salud de las personas. Estos sistemas resultan especialmente útiles para adultos mayores, pacientes con enfermedades crónicas o personas que requieren un seguimiento constante de su estado físico, contribuyendo a mejorar su calidad de vida y facilitando la intervención temprana ante situaciones de riesgo.

Dentro de este contexto surge la necesidad de implementar soluciones accesibles y de bajo costo que permitan la supervisión remota de signos vitales y eventos que puedan comprometer la integridad física de los usuarios. La incorporación de sensores biomédicos y dispositivos conectados a Internet ha dado lugar al desarrollo de sistemas de monitoreo que permiten obtener información relevante y transmitirla a profesionales de la salud o familiares de manera inmediata.

El presente proyecto consiste en el desarrollo de un sistema de monitoreo de salud basado en un microcontrolador ESP32, capaz de medir la frecuencia cardíaca y la saturación de oxígeno en sangre mediante sensores biométricos, así como detectar posibles caídas a través de un sensor acelerómetro. La información recolectada es enviada a una plataforma web desarrollada específicamente para el proyecto, donde los datos pueden visualizarse de forma remota y almacenarse para su posterior análisis.

El sistema busca brindar una herramienta de apoyo para el cuidado de personas que requieren supervisión continua, permitiendo conocer su estado general en tiempo real y generar alertas ante situaciones de emergencia, como una caída. De esta manera, se pretende aprovechar las ventajas de las tecnologías IoT (Internet de las Cosas) para ofrecer una solución práctica, económica y escalable en el ámbito de la salud y el monitoreo remoto.

Abstract

Today, advances in electronic and communications technologies have enabled the development of devices capable of monitoring various health-related parameters in real time. These systems are particularly useful for older adults, patients with chronic conditions, or people who require constant monitoring of their physical condition, helping to improve their quality of life and facilitating early intervention in high-risk situations.

Within this context, there is a need to implement accessible, low-cost solutions that enable remote monitoring of vital signs and events that could compromise users' physical well-being. The integration of biomedical sensors and Internet-connected devices has led to the development of monitoring systems that collect relevant information and transmit it immediately to healthcare professionals or family members.

This project involves the development of a health monitoring system based on an ESP32 microcontroller, capable of measuring heart rate and blood oxygen saturation using biometric sensors, as well as detecting potential falls via an accelerometer sensor. The collected information is sent to a web platform developed specifically for the project, where the data can be viewed remotely and stored for later analysis.

The system aims to provide a support tool for caring for people who require continuous supervision, allowing users to monitor their general condition in real time and generate alerts in emergency situations, such as a fall. In this way, the goal is to leverage the

advantages of IoT (Internet of Things) technologies to offer a practical, cost-effective, and scalable solution in the field of healthcare and remote monitoring.

Dedicatoria

Dedico este trabajo a mis hijas, Camila y Julieta, quienes son mi principal motivación para seguir adelante y afrontar cada nuevo proyecto.

A mis padres, Susana y Pedro, por su apoyo incondicional y por brindarme los medios y el acompañamiento necesarios para poder continuar con mis estudios.

Y a todas las personas que estuvieron presentes durante este camino, acompañándome y alentándome a alcanzar este objetivo.

Agradecimientos

Agradezco a todo el cuerpo docente de la carrera por los conocimientos y las herramientas brindadas a lo largo de mi formación, y especialmente a quienes me acompañaron durante aquellas cursadas que realicé de manera individual, por su dedicación, paciencia y constante predisposición.

También agradezco al Instituto INA por brindarme la oportunidad y los medios necesarios para continuar con la carrera y poder completar esta etapa de mi formación.

Índice

Internet de las Cosas (IoT)	17
Sistemas embebidos	19
ESP32	21
Sensor biométrico MAX30102	25
Sensor MPU6050	28
Comunicación Wi-Fi y protocolo HTTP	32
Comunicación Wi-Fi	32
Protocolo HTTP	33
Formato JSON	34
API REST	37
Node.js y Express	40
Node.js	41
Express	42
Base de datos MySQL	44
Arquitectura Cliente-Servidor	48
Smartwatches	54
Sistemas de monitoreo remoto	56
Sistemas de detección de caídas	56
Comparación con el sistema desarrollado	57
Aspectos diferenciales del proyecto	58
Módulo de adquisición de datos	61
Módulo de comunicación	62
Módulo de procesamiento y almacenamiento	64
Módulo de visualización	64
Diseño del hardware	68
Inicialización del sistema	71
Detección de caídas	76
Desarrollo del Backend	78
Organización del proyecto (Arquitectura basada en MVC)	79
Gestión de usuarios y autenticación	82
Integración del backend con la plataforma web	84
Desarrollo de la Plataforma Web	85
Inicio de sesión	85
Registro de usuarios	87
Monitoreo de pacientes y visualización de registros biométricos	90
Beneficios del monitoreo remoto	94
Resultados obtenidos	94

Aportes del proyecto	97
Limitaciones y trabajos futuros	97
Consideraciones finales	98

Lista de tablas

Tabla 1	22
Características del ESP32	22
Tabla 2	25
Características del sensor MAX30102	25
Tabla 3	28
Características del sensor MPU6050	28
Tabla 4	38
Métodos HTTP principales.	38
Tabla 5	41
Métodos HTTP principales.	41
Tabla 6	45
Métodos HTTP principales.	45
Tabla 7	66
Componentes utilizados en el hardware.	66
Tabla 8	67
Conexiones entre los componentes.	67
Tabla 9	77
Resumen de la función de los principales directorios utilizados durante el desarrollo del backend.	77

Lista de figuras

Figura 1	25
Microcontrolador ESP32	25
Figura 2	29
Sensor biométrico MAX30102 utilizado para la medición de frecuencia cardíaca y saturación de oxígeno en sangre.	29
Figura 3	32
Sensor MPU6050 utilizado para la detección de movimiento y caídas.	32
Figura 4	49
Diagrama entidad relación (DER).	49
Figura 5	53
Sistema de monitoreo remoto, flujo general del sistema.	53
Figura 6	61
Arquitectura general del sistema.	61
Figura 7	62
Diagrama de bloques del hardware.	62
Figura 8	64
Líneas de código donde se organiza la información en formato JSON.	64
Figura 9	67
Página de visualización de datos obtenidos.	67
Figura 10	70
Componentes principales del hardware.	70
Figura 11	75
Proceso de inicialización del sistema.	75
Figura 12	77
Monitor Serie mostrando la inicialización del sistema.	77
Figura 13	79
Líneas de código de la función de detección de caídas.	79
Figura 14	82
Estructura de carpetas del proyecto.	82
Figura 15	84
Diagramas de caso de uso del sistema.	84
Figura 16	88
Pantalla inicio de sesión.	88
Figura 17	90
Pantalla de registro de usuario paciente.	90
Figura 18	91
Pantalla de registro de usuario cuidador.	91

Figura 19	93
Pantalla monitoreo del usuario cuidador.	93

Nomenclatura

ABM	Alta, Baja y Modificación
API	Application Programming Interface
APP	Aplicación
BD	Base de Datos
CRUD	Create, Read, Update and Delete
CSS	Cascading Style Sheets (Hojas de Estilo en Cascada)
CU	Caso de Uso
DAC	Digital to analogue converter (conversor digital analógico)
DER	Diagrama Entidad-Relación
GPS	Global Positioning System (Sistema de Posicionamiento Global)
HTML	HyperText Markup Language (Lenguaje de Marcado de Hipertexto)
IDE	Integrated Development Environment (Entorno de Desarrollo Integrado)
IP	Internet Protocol (Protocolo de Internet)
LED	Light-Emitting Diode (Diodo Emisor de Luz)
MCU	Microcontroller Unit (Unidad de Microcontrolador)
MVC	Model-View-Controller (Modelo-Vista-Controlador)
ORM	Object-Relational Mapping (Mapeo Objeto-Relacional)
SCL	Serial Clock (Señal de reloj del bus I ² C)
SDA	Serial Data (Línea de datos del bus I ² C)
SGBD	Sistema de Gestión de Bases de Datos
SpO ₂	Saturación periférica de oxígeno en sangre
SQL	Structured Query Language (Lenguaje de Consulta Estructurado)
SSID	Service Set Identifier (Identificador de la red Wi-Fi)

UML	Unified Modeling Language (Lenguaje Unificado de Modelado)
URL	Uniform Resource Locator (Localizador Uniforme de Recursos)
USB	Universal Serial Bus (Bus Serie Universal)

Definiciones básicas

Arduino IDE: Entorno de desarrollo que permite escribir, compilar y cargar programas en placas compatibles con Arduino y diferentes microcontroladores. En este proyecto se utiliza para desarrollar y cargar el firmware en el ESP32.

I²C (Inter-Integrated Circuit): Protocolo de comunicación serial utilizado para conectar diferentes dispositivos electrónicos mediante dos líneas principales: SDA, para la transmisión de datos, y SCL, para la señal de reloj. En el proyecto permite la comunicación del ESP32 con los sensores MAX30102 y MPU6050.

Sequelize: Herramienta de mapeo objeto-relacional (ORM) para Node.js que facilita la interacción entre la aplicación y una base de datos relacional. En este proyecto permite definir los modelos y relaciones, así como realizar operaciones sobre la base de datos MySQL.

SpO₂ (Saturación periférica de oxígeno): Valor que representa el porcentaje estimado de hemoglobina de la sangre que transporta oxígeno. En el proyecto constituye uno de los parámetros biométricos obtenidos mediante el sensor MAX30102.

BPM (Beats Per Minute): Unidad utilizada para expresar la frecuencia cardíaca mediante la cantidad de latidos registrados durante un minuto. En el sistema corresponde a uno de los principales valores biométricos monitoreados.

Introducción

El presente Trabajo Final Integrador se realiza con el propósito de obtener el título de Técnico Superior en Diseño y Programación Web, integrando los conocimientos y herramientas adquiridos durante la formación académica mediante el desarrollo de una solución tecnológica aplicada a una problemática concreta.

El proyecto consiste en el diseño y desarrollo de un sistema de monitoreo personal orientado al seguimiento de parámetros biométricos y a la detección de posibles caídas. La propuesta surge a partir de la importancia de contar con herramientas que permitan registrar información sobre el estado de una persona y facilitar su consulta por parte de usuarios autorizados, como familiares o cuidadores.

Para llevar a cabo el sistema se desarrolló un dispositivo basado en un microcontrolador ESP32, conectado a diferentes sensores. El sensor MAX30102 permite obtener información relacionada con la frecuencia cardíaca y la saturación de oxígeno en sangre (SpO_2), mientras que el MPU6050 permite registrar información de movimiento utilizada para identificar posibles eventos de caída.

Los datos obtenidos son procesados por el ESP32 y transmitidos mediante una conexión Wi-Fi hacia una API REST desarrollada con Node.js y Express. El servidor se encarga de recibir, procesar y almacenar la información en una base de datos MySQL, permitiendo mantener un historial de los registros obtenidos.

Como complemento del dispositivo, se desarrolló una plataforma web que permite gestionar usuarios y consultar la información registrada. El sistema contempla diferentes roles, entre ellos pacientes, cuidadores y administradores, permitiendo organizar el acceso a las distintas funcionalidades y visualizar los registros biométricos mediante una interfaz web.

De esta manera, el proyecto integra conceptos de desarrollo web, programación de servidores, bases de datos, comunicación mediante APIs, sistemas embebidos e Internet de las Cosas (IoT). El resultado constituye un prototipo funcional que demuestra la posibilidad de integrar hardware y software dentro de una misma solución de monitoreo, dejando además una arquitectura preparada para futuras ampliaciones y mejoras.

Planteamiento del problema

El envejecimiento de la población y el aumento de enfermedades cardiovasculares y respiratorias han generado la necesidad de contar con herramientas que permitan monitorear el estado de salud de las personas de manera continua. En muchos casos, los adultos mayores o pacientes con determinadas patologías permanecen solos durante gran parte del día, dificultando la detección temprana de situaciones que podrían poner en riesgo su bienestar.

La supervisión periódica de parámetros como la frecuencia cardíaca y la saturación de oxígeno en sangre resulta fundamental para evaluar el estado general de una persona. Sin embargo, los métodos tradicionales requieren la presencia de personal médico o la utilización de equipos especializados que, en algunos casos, poseen costos elevados y no permiten un seguimiento permanente fuera de centros de salud.

Por otra parte, las caídas representan uno de los principales riesgos para los adultos mayores. Una caída puede ocasionar lesiones graves y, si no es detectada rápidamente, aumentar significativamente las consecuencias para la salud del paciente. La ausencia de mecanismos de alerta inmediata dificulta la asistencia oportuna por parte de familiares o cuidadores.

De esta manera, el proyecto busca contribuir al cuidado y supervisión de personas que requieren monitoreo constante, proporcionando una herramienta tecnológica de bajo costo que favorezca una respuesta más rápida ante posibles incidentes y mejore la calidad del seguimiento de su estado de salud.

Objetivos

Desarrollar un sistema de monitoreo remoto de signos vitales basado en un microcontrolador ESP32 que permita medir la frecuencia cardíaca, la saturación de oxígeno en sangre y detectar posibles caídas, enviando la información en tiempo real a una plataforma web para facilitar el seguimiento del estado de salud del usuario y la generación de alertas ante situaciones de emergencia.

Objetivos específicos

1. Analizar las necesidades y requerimientos para el desarrollo de un sistema de monitoreo remoto de signos vitales.
2. Investigar el funcionamiento y las características técnicas de los sensores biométricos y del acelerómetro utilizados en el proyecto.
3. Diseñar la arquitectura de hardware integrando el ESP32 con los sensores de frecuencia cardíaca, saturación de oxígeno y detección de caídas.
4. Desarrollar el firmware del ESP32 para la adquisición, procesamiento y envío de los datos obtenidos por los sensores.
5. Implementar una API REST que permita recibir, almacenar y consultar la información enviada por el dispositivo.
6. Diseñar e implementar una base de datos para el almacenamiento histórico de las mediciones registradas.
7. Desarrollar una plataforma web que permita visualizar en tiempo real los signos vitales del usuario y consultar el historial de mediciones.

8. Implementar un sistema de detección de caídas que genere una alerta cuando se identifique un evento de riesgo.
9. Integrar todos los componentes del sistema (hardware, API, base de datos y plataforma web) para garantizar su correcto funcionamiento.
10. Realizar pruebas de funcionamiento y evaluar la precisión, estabilidad y confiabilidad del sistema desarrollado.
11. Documentar el proceso de desarrollo, implementación y validación del proyecto.

Alcance del proyecto

El presente proyecto comprende el diseño, desarrollo e implementación de un sistema de monitoreo remoto de signos vitales basado en un microcontrolador ESP32, orientado al seguimiento del estado de salud de una persona mediante la medición de parámetros fisiológicos y la detección de posibles caídas.

El sistema desarrollado incluye la integración de sensores capaces de medir la frecuencia cardíaca y la saturación de oxígeno en sangre, así como un acelerómetro utilizado para detectar caídas mediante el análisis del movimiento del usuario. La información obtenida es procesada por el ESP32 y transmitida a través de una red Wi-Fi hacia un servidor, donde es almacenada en una base de datos para su posterior consulta.

Como parte del proyecto también se desarrolló una API REST encargada de recibir y gestionar la información proveniente del dispositivo, además de una plataforma web

que permite visualizar en tiempo real los signos vitales registrados y consultar el historial de mediciones mediante gráficos e indicadores.

Si bien durante la etapa de planificación se consideró la posibilidad de incorporar un módulo GPS para conocer la ubicación del usuario en caso de una caída o emergencia, dicha funcionalidad no fue implementada debido a las limitaciones de tiempo establecidas para el desarrollo de la tesina y al incremento en la complejidad del hardware, el consumo energético y el procesamiento de la información que implicaba su integración.

Asimismo, el sistema no reemplaza equipos médicos profesionales ni está destinado a realizar diagnósticos clínicos. Su finalidad es brindar una herramienta de monitoreo y apoyo que permita obtener información del estado general del usuario y facilitar una respuesta temprana ante posibles situaciones de riesgo.

Marco teórico

El presente capítulo tiene como finalidad describir los conceptos, tecnologías y herramientas que constituyen la base teórica del sistema desarrollado. Se presentan las características de los dispositivos electrónicos, los protocolos de comunicación y las tecnologías de software utilizadas para implementar el sistema de monitoreo remoto de signos vitales.

Comprender estos conceptos resulta fundamental para interpretar el funcionamiento de la solución propuesta, ya que el proyecto integra distintas áreas de la informática y la electrónica, incluyendo sistemas embebidos, Internet de las Cosas (IoT), comunicaciones inalámbricas, desarrollo de aplicaciones web y bases de datos.

Internet de las Cosas (IoT)

Es un paradigma tecnológico que permite la interconexión de objetos físicos mediante Internet, posibilitando que estos recopilen, intercambien y procesen información de manera automática sin necesidad de intervención humana constante.

Un sistema IoT está compuesto, generalmente, por dispositivos electrónicos equipados con sensores y actuadores, una red de comunicaciones, un servidor encargado del procesamiento de la información y una aplicación que permite visualizar o administrar los datos obtenidos.

Gracias a esta tecnología, los dispositivos pueden capturar información del entorno, transmitirla a través de Internet y permitir que otros sistemas la procesen para generar estadísticas, emitir alertas o automatizar determinadas acciones. Estas características han impulsado la adopción del IoT en múltiples sectores, como la industria, la agricultura, la domótica, el transporte y especialmente el área de la salud.

En el ámbito sanitario, el IoT ha favorecido el desarrollo de sistemas de monitoreo remoto capaces de registrar parámetros fisiológicos de los pacientes en tiempo real, facilitando la supervisión continua sin necesidad de permanecer en un establecimiento médico. Este tipo de soluciones permite realizar un seguimiento permanente del estado de salud, detectar situaciones de riesgo de manera temprana y reducir los tiempos de respuesta ante una emergencia.

El sistema desarrollado en el presente Trabajo Final Integrador constituye una aplicación práctica de esta tecnología. El dispositivo basado en el microcontrolador ESP32 adquiere la información proveniente de los sensores biométricos, la procesa y la transmite mediante una conexión Wi-Fi hacia un servidor. Posteriormente, los datos son almacenados en una base de datos y visualizados desde una plataforma web, permitiendo realizar un seguimiento remoto del usuario.

La arquitectura general del proyecto responde al modelo típico de un sistema IoT, compuesto por cuatro etapas principales:

- Captura de datos mediante sensores electrónicos.
- Procesamiento de la información en el microcontrolador ESP32.
- Transmisión de los datos mediante una red inalámbrica hacia un servidor.
- Almacenamiento y visualización de la información mediante una aplicación web.

Esta arquitectura permite desacoplar el hardware del software, facilitando futuras ampliaciones del sistema, como la incorporación de nuevos sensores, algoritmos de análisis o mecanismos de notificación automática.

El uso de tecnologías IoT representa una alternativa eficiente y de bajo costo para desarrollar soluciones de monitoreo remoto, permitiendo integrar dispositivos electrónicos con aplicaciones web capaces de ofrecer información en tiempo real desde cualquier lugar con acceso a Internet.

Sistemas embebidos

Sistema informático diseñado para cumplir una función específica dentro de un dispositivo electrónico. A diferencia de una computadora de propósito general, que puede ejecutar una amplia variedad de aplicaciones, un sistema embebido está dedicado a realizar una o varias tareas determinadas de manera eficiente y con un consumo reducido de recursos.

Generalmente, un sistema embebido está compuesto por un microcontrolador o microprocesador, memoria, dispositivos de entrada y salida, sensores, actuadores y los componentes electrónicos necesarios para interactuar con el entorno. El software que ejecuta suele estar desarrollado específicamente para la función que debe desempeñar el dispositivo, lo que permite optimizar su rendimiento y reducir el consumo de energía.

Los sistemas embebidos se encuentran presentes en numerosos equipos de uso cotidiano, tales como electrodomésticos, automóviles, equipos médicos, sistemas de seguridad, dispositivos industriales y equipos de telecomunicaciones. En los últimos años, su utilización se ha incrementado significativamente debido al crecimiento del Internet de las Cosas (IoT), donde estos dispositivos adquieren información mediante sensores y la transmiten a través de Internet para ser procesada por otros sistemas.

Entre las principales características de los sistemas embebidos se destacan:

- Están diseñados para cumplir una función específica.
- Operan de manera continua y con alta confiabilidad.
- Poseen un tamaño reducido y bajo consumo energético.
- Permiten interactuar con sensores y actuadores.
- Pueden comunicarse con otros dispositivos mediante diferentes protocolos de comunicación, como Wi-Fi, Bluetooth, Ethernet o comunicaciones seriales.

En el presente Trabajo Final Integrador, el sistema embebido está representado por el microcontrolador ESP32, encargado de controlar el funcionamiento de los sensores biométricos, procesar la información obtenida y gestionar la comunicación con el servidor mediante una conexión Wi-Fi.

El firmware desarrollado para el ESP32 permite realizar de forma periódica la lectura de la frecuencia cardíaca, la saturación de oxígeno en sangre y los datos del acelerómetro, procesando esta información antes de enviarla a una API REST para su almacenamiento y posterior visualización en la plataforma web. De esta manera, el sistema embebido constituye el núcleo del dispositivo de monitoreo y el responsable de integrar todos los componentes electrónicos utilizados en el proyecto.

La elección de un sistema embebido basado en ESP32 responde a su capacidad de procesamiento, bajo costo, reducido consumo energético y conectividad inalámbrica integrada, características que lo convierten en una plataforma ampliamente utilizada en proyectos de IoT y monitoreo remoto.

ESP32

El ESP32 es un microcontrolador de bajo costo desarrollado por la empresa Espressif Systems, ampliamente utilizado en proyectos de Internet de las Cosas (IoT), automatización y sistemas embebidos. Su principal característica es la integración de conectividad Wi-Fi y Bluetooth en un único circuito, lo que permite desarrollar dispositivos capaces de comunicarse con otros equipos o con servidores a través de redes inalámbricas sin necesidad de módulos adicionales.

Una de las principales ventajas del ESP32 es la gran variedad de interfaces de comunicación que incorpora por hardware, entre las que se destacan:

- GPIO (General Purpose Input/Output) para entradas y salidas digitales.
- ADC (Conversor Analógico-Digital) para la lectura de señales analógicas.
- PWM (Pulse Width Modulation) para el control de motores, servomotores y regulación de intensidad.
- UART para comunicaciones seriales.
- SPI (Serial Peripheral Interface) para dispositivos de alta velocidad.
- I²C (Inter-Integrated Circuit) para la comunicación con sensores y periféricos electrónicos.

En el presente proyecto se utiliza principalmente la interfaz I²C, ya que tanto el sensor biométrico MAX30102 como el sensor MPU6050 emplean este protocolo de comunicación para intercambiar información con el microcontrolador. La comunicación

se realiza mediante dos líneas de datos: SDA (Serial Data) y SCL (Serial Clock), conectadas a los pines GPIO 21 y GPIO 22 del ESP32, respectivamente.

Otra característica destacada del ESP32 es su conectividad inalámbrica integrada. Gracias al módulo Wi-Fi incorporado, el dispositivo puede conectarse a una red local y establecer comunicación con un servidor remoto utilizando protocolos como HTTP. Esta capacidad elimina la necesidad de utilizar módulos externos de comunicación, simplificando el diseño del hardware y reduciendo los costos del sistema.

Durante el funcionamiento del dispositivo, el ESP32 ejecuta un programa desarrollado en el entorno Arduino IDE. Dicho programa realiza de forma periódica la lectura de los sensores, procesa la información obtenida y construye una estructura de datos en formato JSON. Posteriormente, estos datos son enviados mediante una solicitud HTTP de tipo POST hacia una API REST desarrollada para el proyecto.

Además de las mediciones biométricas, el ESP32 es responsable de controlar la lógica de detección de caídas a partir de la información proporcionada por el acelerómetro MPU6050. Una vez identificada una posible situación de riesgo, el dispositivo transmite el evento junto con el resto de las mediciones para que quede registrado en la plataforma web.

La elección del ESP32 para el desarrollo del sistema se fundamenta en diversas ventajas técnicas. Entre ellas se destacan su elevada capacidad de procesamiento, bajo consumo energético, conectividad Wi-Fi y Bluetooth integrada, amplia disponibilidad de bibliotecas para el manejo de sensores y una extensa comunidad de

desarrolladores que facilita la implementación de nuevos proyectos. Estas características lo convierten en una plataforma ideal para aplicaciones de monitoreo remoto y sistemas basados en Internet de las Cosas.

Tabla 1

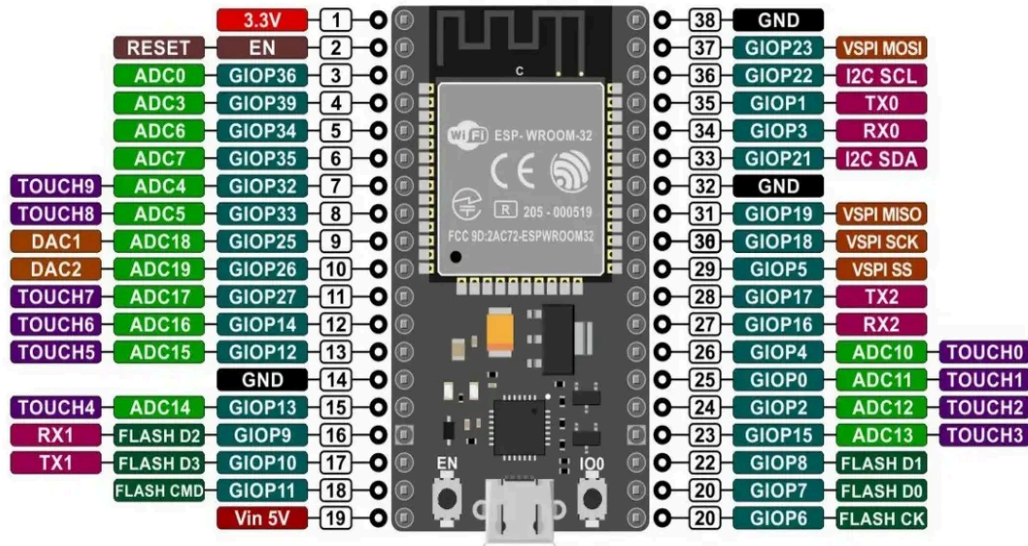
Características del ESP32

Característica	ESP32
Fabricante	Espressif Systems
Arquitectura	Xtensa LX6 de 32 bits
Núcleos	2
Frecuencia	Hasta 240 MHz
Memoria RAM	520 KB
Memoria Flash	4 MB
Wi-Fi	IEEE 802.11 b/g/n
Bluetooth	BLE y Bluetooth Classic
Voltaje de operación	3,3 V
Interfaces	GPIO, ADC, DAC, PWM, UART, SPI, I ² C

Nota: SDA (Serial Data / Datos serie) y SCL (Serial Clock / Reloj Serie).

Figura 1

Microcontrolador ESP32



Sensor biométrico MAX30102

Es un sensor biométrico desarrollado por Maxim Integrated (actualmente perteneciente a Analog Devices), diseñado para aplicaciones de monitoreo de la salud. Este dispositivo integra en un único encapsulado dos diodos emisores de luz (LED), uno de color rojo y otro infrarrojo, junto con un fotodetector de alta sensibilidad y la electrónica necesaria para el acondicionamiento de la señal.

Su principal aplicación consiste en la medición de la frecuencia cardíaca (Heart Rate) y la saturación de oxígeno en sangre (SpO_2) mediante una técnica óptica no invasiva denominada fotoplethysmografía (Photoplethysmography, PPG).

La fotoplethysmografía se basa en iluminar una parte del cuerpo, generalmente la yema de un dedo, con luz de diferentes longitudes de onda. Una parte de esa luz es absorbida por la sangre y otra parte es reflejada hacia el fotodetector. Debido a que el volumen sanguíneo varía con cada latido del corazón, la intensidad de la luz recibida también cambia, permitiendo determinar la frecuencia cardíaca.

Para calcular la saturación de oxígeno en sangre, el sensor utiliza simultáneamente los LED rojo e infrarrojo. La hemoglobina oxigenada y la hemoglobina reducida absorben estas longitudes de onda de manera diferente. Analizando la relación entre ambas señales es posible estimar el porcentaje de oxígeno transportado por la sangre.

La comunicación entre el MAX30102 y el ESP32 se realiza mediante el protocolo I²C, utilizando únicamente dos líneas de comunicación: SDA (Serial Data) y SCL (Serial Clock). Esta interfaz simplifica considerablemente el cableado y permite integrar múltiples dispositivos sobre el mismo bus de comunicación.

En el sistema desarrollado, el ESP32 realiza lecturas periódicas del sensor mediante las bibliotecas correspondientes, obteniendo la frecuencia cardíaca y la saturación de oxígeno del usuario. Estos valores son procesados y posteriormente enviados hacia el servidor mediante una API REST, donde quedan almacenados para su visualización en la plataforma web.

La elección del MAX30102 para este proyecto se debe a su reducido tamaño, bajo consumo energético, facilidad de integración con plataformas de desarrollo como el ESP32 y su amplia utilización en dispositivos portátiles de monitoreo de la salud. Estas

características lo convierten en un sensor adecuado para aplicaciones IoT orientadas al seguimiento remoto de signos vitales.

Tabla 2

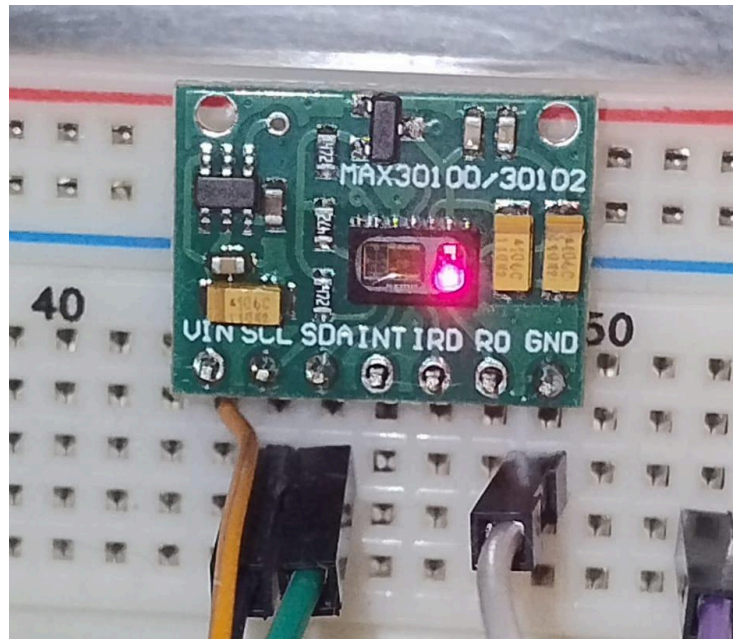
Características del sensor MAX30102

Característica	MAX30102
Función	Medición de frecuencia cardíaca y SpO ₂
Método de medición	Fotopletimografía (PPG)
Comunicación	I ² C
Alimentación	1.8 V (lógica) / 3.3 V (módulos comerciales)
LEDs integrados	Rojo e Infrarrojo
Aplicaciones	Oxímetros, pulseras inteligentes, dispositivos médicos portátiles

Nota: SDA (Serial Data / Datos serie) y SCL (Serial Clock / Reloj Serie).

Figura 2

Sensor biométrico MAX30102 utilizado para la medición de frecuencia cardíaca y saturación de oxígeno en sangre.



Sensor MPU6050

El MPU6050 es un sensor de movimiento desarrollado por InvenSense que integra en un único circuito un acelerómetro de tres ejes y un giroscopio de tres ejes. Esta combinación permite medir tanto la aceleración lineal como la velocidad angular de un objeto, proporcionando información sobre su movimiento y orientación en el espacio.

El acelerómetro mide las aceleraciones que actúan sobre el dispositivo en los tres ejes cartesianos (X, Y y Z). Estas aceleraciones pueden deberse tanto al movimiento del dispositivo como a la fuerza de gravedad. Analizando las variaciones de estos valores

es posible determinar si el dispositivo permanece en reposo, si está siendo desplazado o si experimenta una aceleración repentina.

Por su parte, el giroscopio mide la velocidad angular alrededor de los tres ejes, permitiendo conocer la rotación del dispositivo. La combinación de la información obtenida por ambos sensores proporciona una descripción más completa del movimiento, mejorando la precisión en aplicaciones que requieren conocer la orientación del dispositivo.

La comunicación entre el MPU6050 y el ESP32 se realiza mediante el protocolo I²C, utilizando las líneas SDA (Serial Data) y SCL (Serial Clock), compartiendo el mismo bus de comunicación empleado por el sensor MAX30102. Esta característica simplifica la conexión de ambos dispositivos al microcontrolador y reduce la cantidad de pines necesarios para el sistema.

En el presente proyecto, el MPU6050 es utilizado para detectar posibles caídas del usuario. Para ello, el ESP32 realiza lecturas periódicas de los valores de aceleración correspondientes a los ejes X, Y y Z. A partir de estas mediciones se calcula la magnitud total de la aceleración mediante la combinación de los tres ejes, obteniendo un único valor representativo del movimiento experimentado por el dispositivo.

Cuando la aceleración supera un umbral previamente establecido y se detecta un patrón compatible con una caída, el sistema registra el evento y lo transmite junto con las mediciones de frecuencia cardíaca y saturación de oxígeno hacia el servidor. De

esta manera, la plataforma web puede almacenar el incidente y poner dicha información a disposición del usuario o del cuidador para su posterior análisis.

Si bien existen algoritmos complejos para la detección de caídas que incorporan inteligencia artificial o filtros avanzados de procesamiento de señales, en este proyecto se implementó un método basado en el análisis de la aceleración medida por el sensor. Esta estrategia permitió desarrollar una solución funcional, de bajo costo y adecuada para los objetivos planteados en el Trabajo Final Integrador.

La elección del MPU6050 se fundamenta en su reducido tamaño, bajo consumo energético, facilidad de integración con el ESP32, amplia disponibilidad de bibliotecas de programación y elevada precisión para aplicaciones de monitoreo de movimiento.

Tabla 3

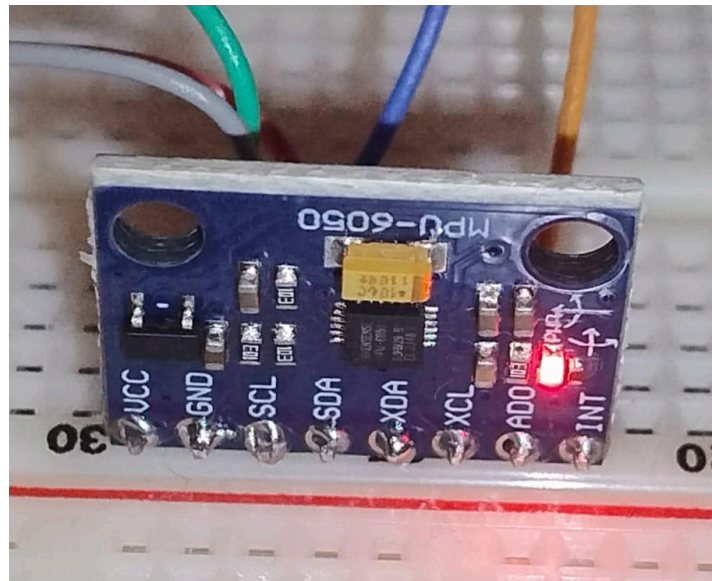
Características del sensor MPU6050

Característica	MAX30102
Función	Acelerómetro y giroscopio de 6 ejes
Método de medición	Fotopletiografía (PPG)
Ejes del acelerómetro	X, Y y Z
Ejes del giroscopio	X, Y y Z
Comunicación	I ² C
Alimentación	3.3 V - 5 V (según el módulo)
Aplicaciones	Detección de movimiento, orientación, robótica, IoT

Nota: SDA (Serial Data / Datos serie) y SCL (Serial Clock / Reloj Serie).

Figura 3

Sensor MPU6050 utilizado para la detección de movimiento y caídas.



Comunicación Wi-Fi y protocolo HTTP

Para que la información obtenida por los sensores pueda ser consultada de forma remota, resulta necesario transmitir los datos desde el dispositivo electrónico hacia un servidor conectado a Internet. En el sistema desarrollado, esta comunicación se realiza mediante una red inalámbrica Wi-Fi y el protocolo HTTP.

Comunicación Wi-Fi

Wi-Fi es una tecnología de comunicación inalámbrica basada en los estándares IEEE 802.11, que permite la transmisión de datos entre dispositivos sin necesidad de utilizar

conexiones físicas. Actualmente constituye uno de los medios de acceso a Internet más utilizados tanto en entornos domésticos como empresariales.

Una de las principales ventajas del ESP32 es que incorpora un módulo Wi-Fi integrado, lo que le permite conectarse directamente a una red inalámbrica sin requerir hardware adicional. Una vez establecida la conexión con el punto de acceso, el microcontrolador puede intercambiar información con servidores ubicados dentro de la misma red local o en Internet.

En el presente proyecto, el ESP32 se conecta a una red Wi-Fi previamente configurada mediante el nombre de la red (SSID) y su contraseña. Una vez establecida la conexión, el dispositivo permanece comunicado con el servidor, permitiendo enviar periódicamente las mediciones obtenidas por los sensores biométricos.

Esta capacidad de comunicación inalámbrica elimina la necesidad de utilizar cables para la transmisión de información, facilitando el uso del dispositivo en distintos entornos y permitiendo que el monitoreo pueda realizarse de forma remota.

Protocolo HTTP

El Protocolo de Transferencia de Hipertexto (HTTP, HyperText Transfer Protocol) es uno de los protocolos de comunicación más utilizados en Internet. Su función principal consiste en permitir el intercambio de información entre un cliente y un servidor mediante un modelo de solicitud y respuesta.

En este modelo, el cliente envía una petición al servidor solicitando una determinada operación, mientras que el servidor procesa dicha solicitud y devuelve una respuesta indicando el resultado de la operación realizada.

HTTP define diferentes métodos de comunicación, entre los que se destacan:

- GET: utilizado para solicitar información almacenada en el servidor.
- POST: empleado para enviar nuevos datos al servidor.
- PUT: utilizado para actualizar información existente.
- DELETE: destinado a eliminar registros previamente almacenados.

En el sistema desarrollado, el ESP32 actúa como cliente HTTP. Cada vez que obtiene una nueva medición de frecuencia cardíaca, saturación de oxígeno o detecta una posible caída, construye una solicitud HTTP de tipo POST y la envía hacia la API REST implementada en el servidor.

La información viaja a través de la red utilizando el protocolo HTTP y es recibida por la aplicación desarrollada con Node.js y Express, donde posteriormente es procesada y almacenada en la base de datos.

La utilización de HTTP presenta diversas ventajas para este tipo de aplicaciones, entre ellas su simplicidad, amplia compatibilidad con diferentes plataformas y facilidad de integración con servicios web. Estas características convierten a este protocolo en una de las alternativas más utilizadas para la comunicación entre dispositivos IoT y aplicaciones desarrolladas sobre tecnologías web.

Formato JSON

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos ampliamente utilizado para la comunicación entre aplicaciones y servicios web. Su principal característica es que permite representar información de manera estructurada utilizando una sintaxis sencilla y fácil de interpretar tanto por las personas como por los sistemas informáticos.

Originalmente derivado del lenguaje JavaScript, JSON se ha convertido en un estándar independiente utilizado por la mayoría de los lenguajes de programación y plataformas de desarrollo debido a su simplicidad, bajo tamaño y facilidad de procesamiento.

La información en formato JSON se organiza mediante pares de clave y valor (key-value), donde cada dato posee un nombre que lo identifica y un valor asociado. Además, permite agrupar datos relacionados utilizando objetos y arreglos, facilitando el intercambio de estructuras de información más complejas.

Un ejemplo de un objeto JSON utilizado en el presente proyecto es el siguiente:

```
{  
  "device_serial": "ESP32-001",  
  "heart_rate": 76,  
  "blood_oxygen": 98,  
  "fall_detected": false,  
  "acc_x": 0.12,  
  "acc_y": -0.08,  
  "acc_z": 9.81  
}
```

En este ejemplo, cada medición realizada por el dispositivo se encuentra identificada mediante una clave descriptiva, permitiendo que el servidor interprete correctamente el significado de cada dato recibido.

En el sistema desarrollado, el ESP32 genera una estructura JSON cada vez que realiza una nueva lectura de los sensores. Posteriormente, este objeto es enviado mediante una solicitud HTTP de tipo POST hacia la API REST implementada en el servidor.

Una vez recibido el mensaje, la API procesa automáticamente la información contenida en el objeto JSON, valida los datos y los almacena en la base de datos para su posterior consulta desde la plataforma web.

El uso de JSON aporta numerosas ventajas dentro de este tipo de aplicaciones. Entre las más importantes pueden mencionarse:

- Sintaxis sencilla y fácil de interpretar.
- Bajo tamaño de los mensajes transmitidos.
- Compatibilidad con prácticamente todos los lenguajes de programación.
- Facilidad para convertir la información en objetos dentro de las aplicaciones.
- Excelente integración con APIs REST y aplicaciones web.

En el presente Trabajo Final Integrador, JSON constituye el medio de intercambio de información entre el dispositivo basado en ESP32 y la plataforma desarrollada con Node.js. Gracias a este formato, las mediciones obtenidas por los sensores pueden ser transmitidas, procesadas y almacenadas de manera organizada, garantizando una comunicación eficiente entre todos los componentes del sistema.

API REST

Una API (Application Programming Interface o Interfaz de Programación de Aplicaciones) es un conjunto de reglas y mecanismos que permiten la comunicación entre diferentes aplicaciones o sistemas informáticos. Su función principal consiste en establecer una forma estandarizada para que un programa pueda solicitar información, enviar datos o acceder a determinados servicios ofrecidos por otro programa.

Las API son ampliamente utilizadas en el desarrollo de aplicaciones modernas, ya que permiten separar las distintas partes de un sistema y facilitar la comunicación entre ellas. Gracias a esta arquitectura, un mismo servidor puede proporcionar información a diferentes clientes, como aplicaciones web, aplicaciones móviles o dispositivos IoT.

Dentro de los distintos tipos de API existentes, uno de los estilos arquitectónicos más utilizados es REST (Representational State Transfer). Este modelo define una serie de principios para el intercambio de información utilizando el protocolo HTTP, permitiendo desarrollar servicios simples, escalables y fácilmente integrables con otras aplicaciones.

Una API REST organiza la información mediante recursos, los cuales pueden representar usuarios, dispositivos, productos, mediciones u otros elementos del sistema. Cada recurso posee una dirección única (URL o endpoint) mediante la cual puede ser consultado o modificado.

La comunicación entre el cliente y el servidor se realiza utilizando los diferentes métodos definidos por el protocolo HTTP:

- GET: permite obtener información almacenada en el servidor.
- POST: se utiliza para crear nuevos registros enviando información al servidor.
- PUT: permite modificar datos previamente almacenados.
- DELETE: elimina registros existentes.

En la mayoría de las aplicaciones modernas, las API REST utilizan el formato JSON para intercambiar información, debido a su simplicidad y facilidad de procesamiento por parte de distintos lenguajes de programación.

En el presente Trabajo Final Integrador, la API REST constituye el punto de comunicación entre el dispositivo basado en ESP32, la base de datos y la plataforma web. El microcontrolador actúa como cliente, enviando periódicamente las mediciones obtenidas por los sensores mediante solicitudes HTTP de tipo POST.

Al recibir una solicitud, la API valida la información enviada por el dispositivo, verifica que los datos sean correctos y posteriormente los almacena en la base de datos. De esta forma, cada medición queda registrada y disponible para ser consultada desde la aplicación web.

Además de recibir información proveniente del ESP32, la API también permite que la plataforma web consulte los datos almacenados mediante solicitudes HTTP de tipo GET. Gracias a este mecanismo, los usuarios pueden visualizar el historial de mediciones, administrar dispositivos registrados y acceder a la información almacenada sin necesidad de comunicarse directamente con la base de datos.

La utilización de una API REST aporta numerosas ventajas al sistema desarrollado.

Entre las más importantes pueden destacarse:

- Independencia entre el hardware y la aplicación web.
- Facilidad para incorporar nuevos dispositivos en el futuro.
- Posibilidad de reutilizar la misma API desde aplicaciones móviles o de escritorio.
- Mejor organización del código mediante una arquitectura cliente-servidor.
- Mayor escalabilidad y facilidad de mantenimiento.

En el desarrollo del proyecto, la API REST fue implementada utilizando Node.js y el framework Express, encargándose de recibir las solicitudes provenientes del ESP32, procesar la información y gestionar el acceso a la base de datos. De esta manera, la API actúa como el núcleo de comunicación del sistema, integrando todos los componentes que forman parte de la solución propuesta.

Tabla 4

Métodos HTTP principales.

Método HTTP	Función	Uso en el proyecto
GET	Obtener información	Consultar usuarios, dispositivos y mediciones desde la plataforma web
POST	Crear nuevos registros	Enviar mediciones del ESP32 al servidor
PUT	Actualizar información	Modificar datos de usuarios o dispositivos
DELETE	Eliminar registros	Eliminar usuarios, dispositivos o mediciones cuando corresponda

Nota: SDA (Serial Data / Datos serie) y SCL (Serial Clock / Reloj Serie).

Node.js y Express

Para desarrollar el servidor encargado de procesar la información enviada por el dispositivo de monitoreo se utilizó Node.js junto con el framework Express. Estas tecnologías constituyen una de las combinaciones más utilizadas actualmente para el desarrollo de aplicaciones web y APIs REST, debido a su rendimiento, flexibilidad y facilidad de integración con diferentes bases de datos y servicios.

Node.js

Node.js es un entorno de ejecución que permite utilizar el lenguaje JavaScript fuera del navegador, posibilitando el desarrollo de aplicaciones del lado del servidor. Fue creado con el objetivo de construir aplicaciones escalables capaces de atender múltiples solicitudes simultáneamente utilizando un modelo de ejecución basado en eventos y operaciones de entrada y salida no bloqueantes (Event-Driven y Non-Blocking I/O).

A diferencia de los servidores tradicionales, donde cada solicitud puede ocupar un hilo de ejecución independiente, Node.js utiliza un único hilo principal que administra múltiples conexiones de manera eficiente. Este modelo reduce el consumo de recursos del sistema y mejora el rendimiento cuando existen numerosos clientes comunicándose al mismo tiempo.

Estas características hacen que Node.js sea especialmente adecuado para aplicaciones IoT, donde múltiples dispositivos pueden enviar información periódicamente hacia un mismo servidor sin afectar significativamente su desempeño.

En el presente proyecto, Node.js constituye la base del servidor encargado de recibir las mediciones enviadas por el ESP32, procesarlas y almacenarlas en la base de datos. Asimismo, proporciona la lógica necesaria para responder a las solicitudes realizadas desde la plataforma web.

Express

Express es un framework de código abierto desarrollado sobre Node.js que facilita la creación de aplicaciones web y APIs REST. Proporciona un conjunto de herramientas que simplifican la definición de rutas, el procesamiento de solicitudes HTTP y la organización del código del servidor.

Una de las principales ventajas de Express es su estructura modular, permitiendo separar las distintas responsabilidades de la aplicación en controladores, rutas, modelos y servicios. Esta organización mejora la legibilidad del código, facilita el mantenimiento y favorece el crecimiento del proyecto.

En el desarrollo del sistema de monitoreo, Express fue utilizado para implementar la API REST que recibe la información enviada por el ESP32 y atiende las solicitudes provenientes de la plataforma web. Cada funcionalidad del sistema se encuentra asociada a una ruta específica, permitiendo gestionar de forma organizada las diferentes operaciones disponibles.

Entre las principales funciones desarrolladas mediante Express se encuentran:

- Recepción de las mediciones enviadas por el dispositivo ESP32.
- Validación de la información recibida antes de su almacenamiento.
- Gestión de usuarios, dispositivos y registros biométricos.
- Consulta del historial de mediciones almacenadas en la base de datos.
- Respuesta a las solicitudes realizadas desde la plataforma web mediante datos en formato JSON.

La integración entre Node.js y Express permitió desarrollar un servidor robusto y flexible, capaz de actuar como intermediario entre el hardware y la aplicación web. Mientras el ESP32 envía las mediciones utilizando solicitudes HTTP, Express recibe dichas solicitudes, ejecuta la lógica correspondiente y realiza las operaciones necesarias sobre la base de datos.

De esta manera, Node.js y Express constituyen el núcleo del backend del sistema, siendo responsables de coordinar la comunicación entre todos los componentes que forman parte de la solución desarrollada. Su utilización permitió implementar una arquitectura organizada, escalable y fácilmente extensible para futuras mejoras del proyecto.

Tabla 5

Métodos HTTP principales.

Tecnología	Función en el proyecto
Node.js	Entorno de ejecución del servidor backend.
Express	Framework utilizado para desarrollar la API REST.
HTTP	Protocolo utilizado para la comunicación entre el ESP32 y el servidor.
JSON	Formato utilizado para intercambiar información entre cliente y servidor.

Nota: SDA (Serial Data / Datos serie) y SCL (Serial Clock / Reloj Serie).

Base de datos MySQL

Una base de datos es un conjunto organizado de información que permite almacenar, consultar, modificar y administrar datos de manera eficiente. En aplicaciones informáticas, las bases de datos constituyen uno de los componentes fundamentales, ya que permiten conservar la información generada por los usuarios o por los distintos dispositivos que forman parte de un sistema.

En el presente Trabajo Final Integrador se utilizó MySQL como Sistema Gestor de Bases de Datos Relacional (RDBMS, Relational Database Management System). MySQL es una de las plataformas más utilizadas para el desarrollo de aplicaciones web debido a su estabilidad, rendimiento, facilidad de administración y compatibilidad con diferentes lenguajes de programación.

Las bases de datos relacionales organizan la información en tablas compuestas por filas y columnas. Cada tabla almacena información correspondiente a una entidad específica del sistema, mientras que las relaciones entre ellas permiten mantener la integridad y consistencia de los datos.

En el sistema desarrollado, la base de datos almacena toda la información necesaria para el funcionamiento de la plataforma de monitoreo. Entre los principales datos registrados se encuentran:

- Información de los usuarios del sistema.

- Dispositivos de monitoreo asociados a cada usuario.
- Historial de mediciones biométricas.
- Eventos de detección de caídas.
- Enfermedades registradas para cada paciente.

Cada medición enviada por el ESP32 es recibida por la API REST y almacenada como un nuevo registro dentro de la base de datos. De esta manera, el sistema conserva un historial cronológico de las mediciones realizadas, permitiendo consultar la evolución de los signos vitales y los eventos de caída registrados durante el monitoreo.

El acceso a la base de datos es gestionado por el servidor desarrollado con Node.js, el cual se encarga de realizar las operaciones de consulta, inserción, actualización y eliminación de información según las solicitudes recibidas desde la plataforma web o desde el dispositivo de monitoreo.

Entre las principales ventajas de utilizar MySQL en este proyecto pueden destacarse:

- Organización estructurada de la información.
- Integridad y consistencia de los datos almacenados.
- Rapidez en las consultas y operaciones sobre grandes volúmenes de información.
- Facilidad para establecer relaciones entre distintas entidades del sistema.
- Compatibilidad con tecnologías web modernas como Node.js y Express.

- Escalabilidad para incorporar nuevos módulos y funcionalidades en futuras versiones del sistema.

La implementación de una base de datos relacional permitió disponer de un repositorio centralizado para toda la información generada por el sistema de monitoreo, facilitando tanto el almacenamiento histórico de las mediciones como la consulta de los datos desde la plataforma web.

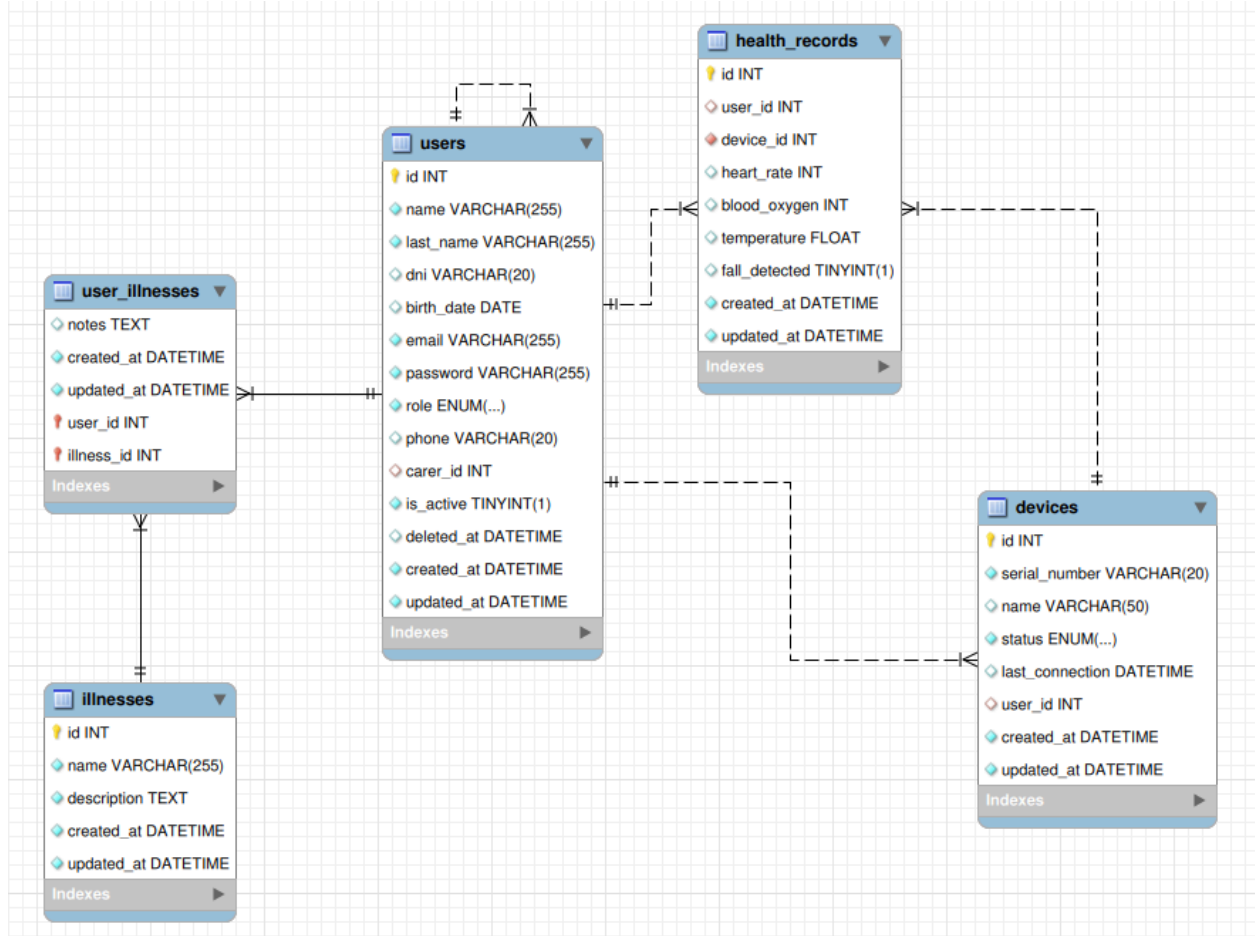
Tabla 6

Métodos HTTP principales.

Entidad	Descripción
Usuarios	Almacena la información de los pacientes, cuidadores y administradores.
Dispositivos	Registra los dispositivos ESP32 asociados a cada usuario.
Registros de salud	Guarda las mediciones de frecuencia cardíaca, saturación de oxígeno, temperatura, aceleración y detección de caídas.
Enfermedades	Contiene el catálogo de enfermedades registradas.
Usuario_Enfermedades	Relaciona los usuarios con una o más enfermedades.

Figura 4

Diagrama entidad relación (DER).



Arquitectura Cliente-Servidor

La arquitectura Cliente-Servidor es un modelo de diseño ampliamente utilizado en el desarrollo de aplicaciones informáticas y servicios web. Este modelo divide el sistema en dos componentes principales: el cliente, encargado de solicitar información o servicios, y el servidor, responsable de procesar dichas solicitudes, acceder a los datos y generar una respuesta.

Esta separación de responsabilidades permite desarrollar sistemas más organizados, escalables y fáciles de mantener, ya que cada componente puede evolucionar de manera independiente sin afectar el funcionamiento del resto del sistema.

En una arquitectura Cliente-Servidor, el cliente no accede directamente a la base de datos. En su lugar, toda la comunicación se realiza a través del servidor, que actúa como intermediario entre los diferentes componentes de la aplicación. De esta forma, el servidor puede validar la información recibida, aplicar las reglas de negocio y garantizar la integridad de los datos almacenados.

En el presente Trabajo Final Integrador, la arquitectura Cliente-Servidor constituye la base del funcionamiento del sistema de monitoreo remoto. Cada uno de los componentes desarrollados cumple una función específica dentro de esta arquitectura.

El dispositivo basado en el ESP32 actúa como cliente IoT. Su función consiste en adquirir las mediciones provenientes del sensor biométrico MAX30102 y del sensor de

movimiento MPU6050, procesar la información y enviarla periódicamente al servidor mediante solicitudes HTTP utilizando datos en formato JSON.

El servidor fue desarrollado utilizando Node.js y Express, implementando una API REST encargada de recibir las solicitudes provenientes del dispositivo. Una vez recibida la información, el servidor realiza las validaciones necesarias y ejecuta las operaciones correspondientes sobre la base de datos MySQL.

La base de datos constituye el repositorio central del sistema, donde se almacenan los usuarios registrados, los dispositivos asociados y el historial de mediciones biométricas. Gracias a esta organización, es posible conservar un registro cronológico de toda la información generada durante el monitoreo.

Finalmente, la plataforma web actúa como un segundo cliente dentro de la arquitectura. A diferencia del ESP32, que envía información al servidor, la aplicación web consulta los datos almacenados mediante solicitudes HTTP a la API REST, permitiendo visualizar las mediciones, administrar usuarios y acceder al historial registrado en la base de datos.

La interacción entre todos estos componentes puede resumirse en las siguientes etapas:

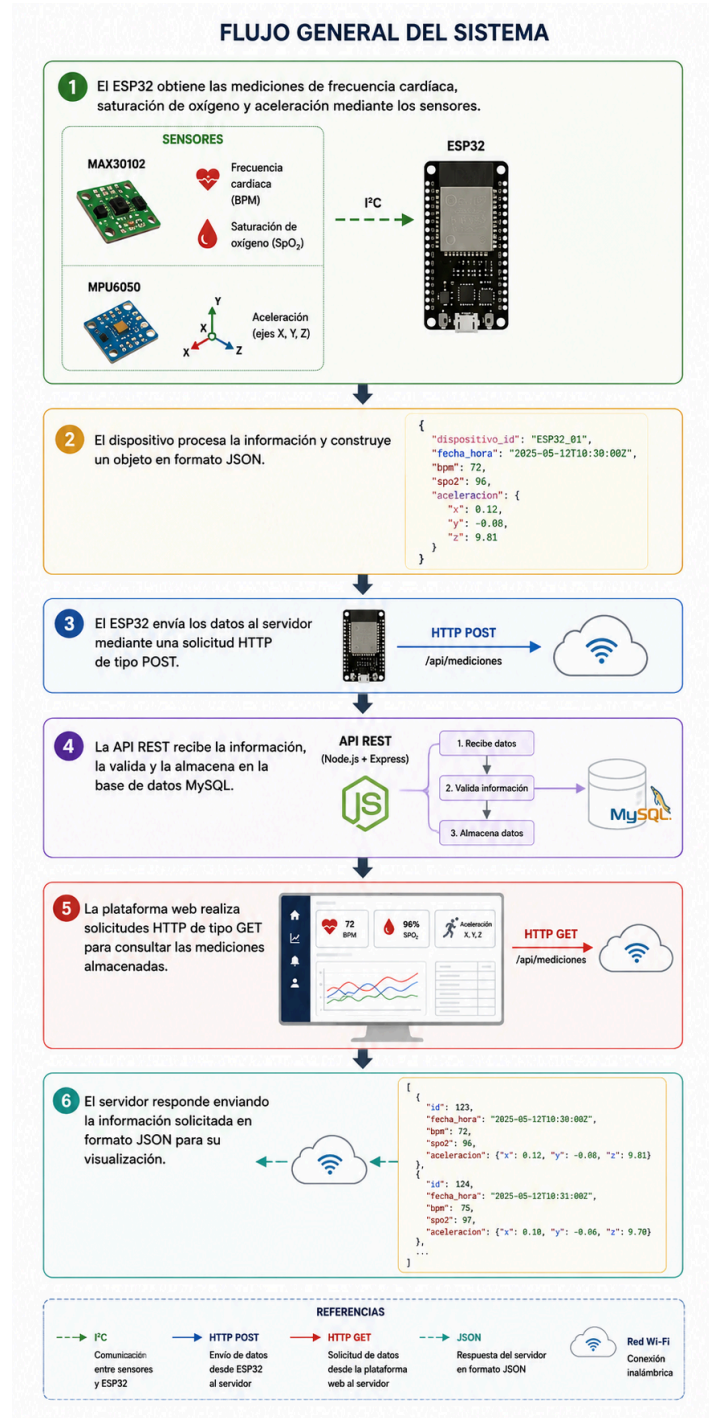
El ESP32 obtiene las mediciones de frecuencia cardíaca, saturación de oxígeno y aceleración mediante los sensores.

- El dispositivo procesa la información y construye un objeto en formato JSON.
- El ESP32 envía los datos al servidor mediante una solicitud HTTP de tipo POST.
- La API REST recibe la información, la valida y la almacena en la base de datos MySQL.
- La plataforma web realiza solicitudes HTTP de tipo GET para consultar las mediciones almacenadas.
- El servidor responde enviando la información solicitada en formato JSON para su visualización.

La implementación de una arquitectura Cliente-Servidor permitió separar claramente las responsabilidades de cada componente del sistema, facilitando su desarrollo, mantenimiento y futura ampliación. Asimismo, esta estructura posibilita incorporar nuevos dispositivos de monitoreo, aplicaciones móviles u otros servicios sin necesidad de modificar el funcionamiento del hardware existente, convirtiendo al sistema en una solución flexible y escalable para futuras mejoras.

Figura 5

Sistema de monitoreo remoto, flujo general del sistema.



Historia del Arte

En los últimos años, el desarrollo de dispositivos electrónicos portátiles y tecnologías asociadas al Internet de las Cosas (IoT) ha permitido la aparición de diferentes soluciones destinadas al monitoreo de la actividad física y determinados parámetros relacionados con la salud. Entre las alternativas disponibles actualmente se encuentran los relojes inteligentes, las pulseras de actividad y diferentes sistemas de monitoreo remoto.

Estos dispositivos incorporan sensores capaces de registrar información del usuario y almacenarla o transmitirla hacia aplicaciones externas. Algunas soluciones permiten además identificar determinadas situaciones de emergencia, como una caída, y generar alertas.

A continuación, se describen algunas de las tecnologías existentes relacionadas con el sistema desarrollado en el presente proyecto.

Smartwatches

Los relojes inteligentes o smartwatches constituyen una de las soluciones comerciales más extendidas para el monitoreo personal. Estos dispositivos incorporan diferentes sensores dentro de un equipo compacto que puede utilizarse continuamente en la muñeca.

Dependiendo del fabricante y del modelo, pueden registrar parámetros como frecuencia cardíaca, saturación de oxígeno, actividad física, sueño, temperatura y otras métricas relacionadas con el bienestar del usuario.

Un ejemplo es el Apple Watch, que permite realizar un seguimiento de la frecuencia cardíaca mediante sensores ópticos y, en determinados modelos y regiones, obtener mediciones de saturación de oxígeno. Algunos modelos también incorporan detección de caídas fuertes, pudiendo mostrar una alerta y comunicarse con servicios de emergencia cuando se cumplen determinadas condiciones.

Otro ejemplo corresponde a los dispositivos Samsung Galaxy Watch, que integran diferentes funcionalidades orientadas al seguimiento de la salud y actividad física. Dependiendo del modelo, permiten registrar frecuencia cardíaca, saturación de oxígeno, electrocardiograma y otros parámetros. Algunos modelos también incorporan mecanismos de detección de caídas y comunicación con contactos de emergencia.

Los relojes Garmin constituyen otra alternativa dentro de este mercado. Diferentes modelos incorporan sensores ópticos para registrar frecuencia cardíaca y estimar la saturación periférica de oxígeno, además de numerosas funciones relacionadas con actividad física y entrenamiento.

Estos dispositivos comerciales presentan un alto nivel de integración, portabilidad y facilidad de utilización. Sin embargo, normalmente funcionan dentro de ecosistemas tecnológicos definidos por cada fabricante, utilizando aplicaciones, servicios y mecanismos de almacenamiento propios. Además, algunas funcionalidades pueden depender del modelo adquirido, del teléfono utilizado o de la disponibilidad del servicio en cada región.

Sistemas de monitoreo remoto

Además de los relojes inteligentes, existen sistemas específicamente diseñados para realizar un seguimiento remoto de personas. Estas soluciones pueden estar compuestas por sensores portátiles, dispositivos instalados en el hogar, aplicaciones móviles y plataformas de almacenamiento de información.

Este tipo de arquitectura resulta especialmente relevante para el cuidado de personas que requieren supervisión, ya que permite separar físicamente el dispositivo encargado de obtener la información del sistema utilizado para visualizarla.

Sistemas de detección de caídas

La detección automática de caídas constituye otra tecnología relacionada directamente con el presente proyecto. Estos sistemas utilizan principalmente acelerómetros y, en algunos casos, giroscopios para analizar los movimientos realizados por una persona.

Esta tecnología se encuentra incorporada en algunos dispositivos comerciales, entre ellos determinados relojes inteligentes. Sin embargo, la detección presenta un desafío importante: diferenciar una caída real de movimientos cotidianos que generan aceleraciones elevadas. Por esta razón, ningún sistema basado exclusivamente en este tipo de sensores puede garantizar la identificación correcta de todas las situaciones.

Comparación con el sistema desarrollado

Las soluciones analizadas demuestran que actualmente existe una tendencia hacia la integración de sensores biométricos, conectividad inalámbrica y plataformas digitales para realizar un seguimiento del estado de las personas.

El sistema desarrollado en este Trabajo Final Integrador comparte algunos principios de funcionamiento con estos dispositivos, principalmente la adquisición de parámetros biométricos y de movimiento mediante sensores y la posibilidad de consultar posteriormente la información obtenida.

Sin embargo, el objetivo del proyecto no consiste en competir directamente con dispositivos comerciales como los smartwatches, los cuales poseen un nivel de miniaturización, precisión e integración considerablemente superior. El propósito consiste en desarrollar e integrar una arquitectura propia que permita comprender y controlar las diferentes etapas involucradas en un sistema IoT de monitoreo.

Para ello, el prototipo utiliza un ESP32 como unidad central, un sensor MAX30102 para obtener información relacionada con la frecuencia cardíaca y saturación de oxígeno, y un MPU6050 para analizar el movimiento y detectar posibles caídas.

A diferencia de una solución comercial cerrada, el sistema implementado integra estos componentes con una API REST desarrollada específicamente para el proyecto, una base de datos MySQL y una plataforma web propia.

Otra característica del sistema es la incorporación de diferentes tipos de usuarios. La plataforma permite gestionar pacientes, cuidadores y administradores, estableciendo relaciones entre ellos y permitiendo que los usuarios autorizados consulten los registros biométricos correspondientes.

Aspectos diferenciales del proyecto

El principal aporte del proyecto no se encuentra en la invención de un nuevo sensor o método de medición, dado que las tecnologías utilizadas ya existen y se encuentran disponibles comercialmente. Su aspecto diferencial reside en la integración de diferentes tecnologías dentro de una arquitectura propia y adaptable.

El sistema combina en una única solución:

- Medición de frecuencia cardíaca.
- Estimación de saturación de oxígeno en sangre.
- Detección de posibles caídas.
- Comunicación inalámbrica mediante Wi-Fi.
- Transmisión de información utilizando HTTP y JSON.
- API REST propia.
- Almacenamiento de registros biométricos en una base de datos.
- Plataforma web para consulta y visualización de las mediciones.
- Gestión diferenciada de pacientes, cuidadores y administradores.

Por lo tanto, la característica diferencial del proyecto se encuentra principalmente en haber desarrollado una solución integral y modular de monitoreo, construida a partir de tecnologías accesibles y cuyo funcionamiento puede ser adaptado y ampliado según las necesidades futuras.

Arquitectura del sistema

La arquitectura del sistema fue diseñada siguiendo el modelo Cliente-Servidor, integrando componentes de hardware y software que trabajan de forma coordinada para realizar el monitoreo remoto de signos vitales. El objetivo principal de esta arquitectura es permitir la adquisición de información biométrica, su transmisión mediante una red inalámbrica, el almacenamiento de los datos y su posterior visualización desde una plataforma web.

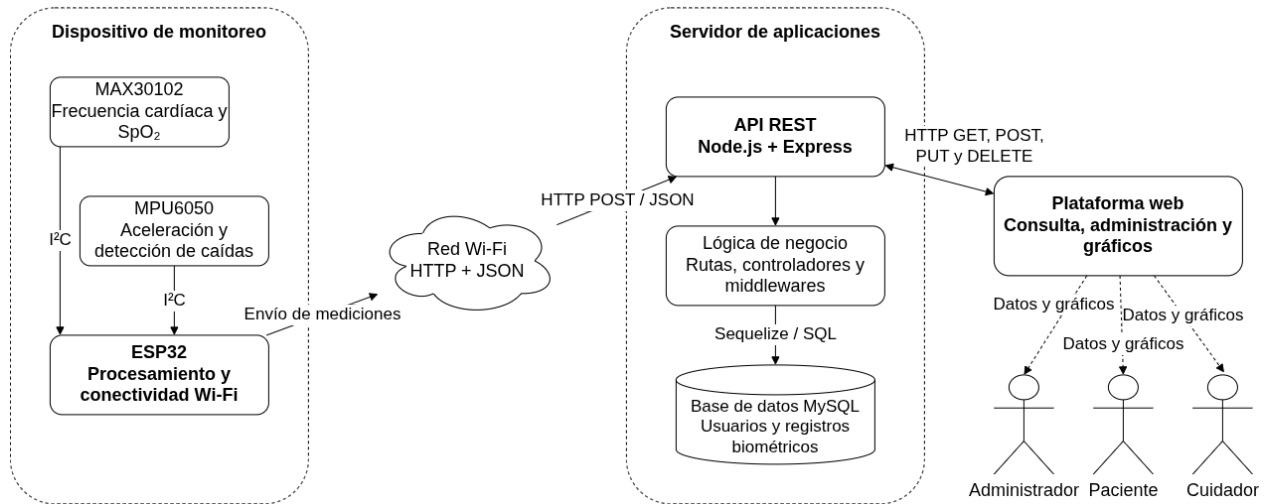
El sistema se encuentra dividido en cuatro módulos principales:

- Módulo de adquisición de datos.
- Módulo de comunicación.
- Módulo de procesamiento y almacenamiento.
- Módulo de visualización.

Cada uno de estos módulos cumple una función específica dentro del funcionamiento general del proyecto y mantiene comunicación con los demás componentes mediante protocolos estandarizados.

Figura 6

Arquitectura general del sistema.



Nota: Diagrama de arquitectura general, mostrando la interacción entre los sensores, el ESP32, la red Wi-Fi, la API REST, la base de datos y la plataforma web.

Módulo de adquisición de datos

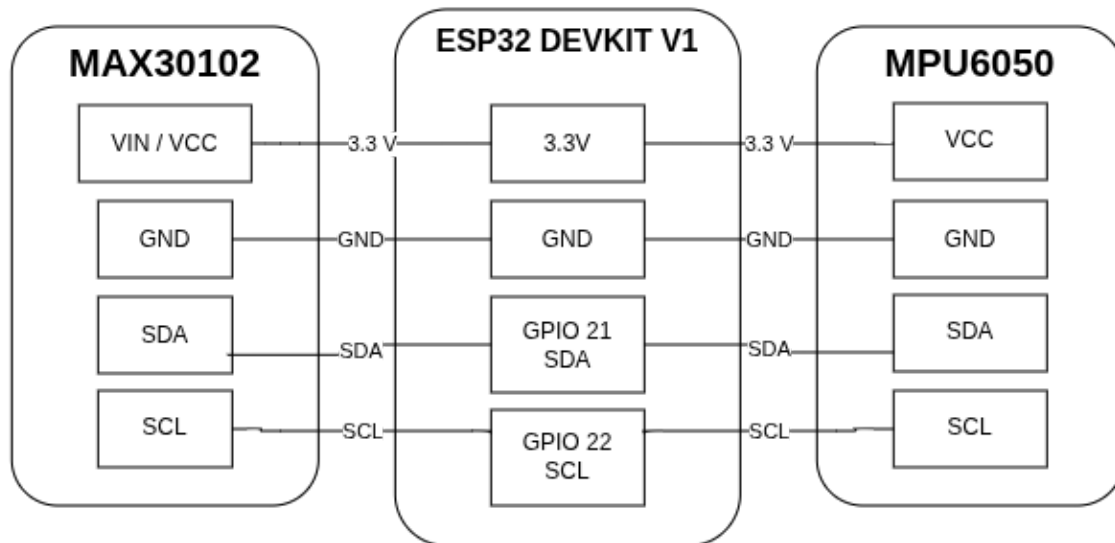
Este módulo corresponde al dispositivo electrónico encargado de obtener la información del usuario. Está compuesto por el microcontrolador ESP32, el sensor biométrico MAX30102 y el sensor de movimiento MPU6050.

El MAX30102 realiza la medición de la frecuencia cardíaca y de la saturación de oxígeno en sangre (SpO₂), mientras que el MPU6050 registra los valores de aceleración y orientación necesarios para detectar posibles caídas.

El ESP32 controla el funcionamiento de ambos sensores, realiza las lecturas periódicas y procesa la información antes de transmitirla al servidor.

Figura 7

Diagrama de bloques del hardware.



Nota: Diagrama de bloques del dispositivo, indicando las conexiones entre el ESP32, el MAX30102 y el MPU6050 mediante el bus de comunicación I²C.

Módulo de comunicación

Una vez obtenidas las mediciones, el ESP32 establece una conexión con la red Wi-Fi previamente configurada y transmite la información hacia el servidor utilizando el protocolo HTTP.

Los datos son organizados en formato JSON y enviados mediante solicitudes HTTP de tipo POST a la API REST desarrollada para el proyecto.

Figura 8

Líneas de código donde se organiza la información en formato JSON.

```
// =====  
// CREAR JSON CON ARDUINOJSON  
// =====  
  
StaticJsonDocument<256> doc;  
  
doc["device_serial"] = deviceSerial;  
doc["heart_rate"] = (beatAvg == 0) ? -1 : beatAvg;  
doc["blood_oxygen"] = (bloodOxygen == 0) ? -1 : bloodOxygen;  
doc["temperature"] = nullptr;  
doc["fall_detected"] = fallDetected;  
  
doc["acc_x"] = accX;  
doc["acc_y"] = accY;  
doc["acc_z"] = accZ;  
  
doc["acc_magnitude"] = accMagnitude;  
  
String jsonData;  
  
serializeJson(doc, jsonData);  
  
Serial.println("JSON enviado:");  
Serial.println(jsonData);  
  
int httpResponseCode =  
  http.POST(jsonData);  
  
Serial.print("HTTP Code: ");  
Serial.println(httpResponseCode);  
  
http.end();  
}
```


Este mecanismo permite desacoplar el dispositivo físico del servidor, facilitando futuras ampliaciones e integraciones con otros sistemas.

Módulo de procesamiento y almacenamiento

El servidor, desarrollado utilizando Node.js y Express, recibe las solicitudes enviadas por el ESP32, valida la información recibida y ejecuta las operaciones necesarias para almacenarla en la base de datos MySQL.

Además del almacenamiento de las mediciones, el servidor administra los usuarios registrados, los dispositivos asociados y las consultas realizadas desde la plataforma web.

La utilización de una API REST permite centralizar toda la lógica del sistema y garantizar una comunicación organizada entre los diferentes componentes.

Módulo de visualización

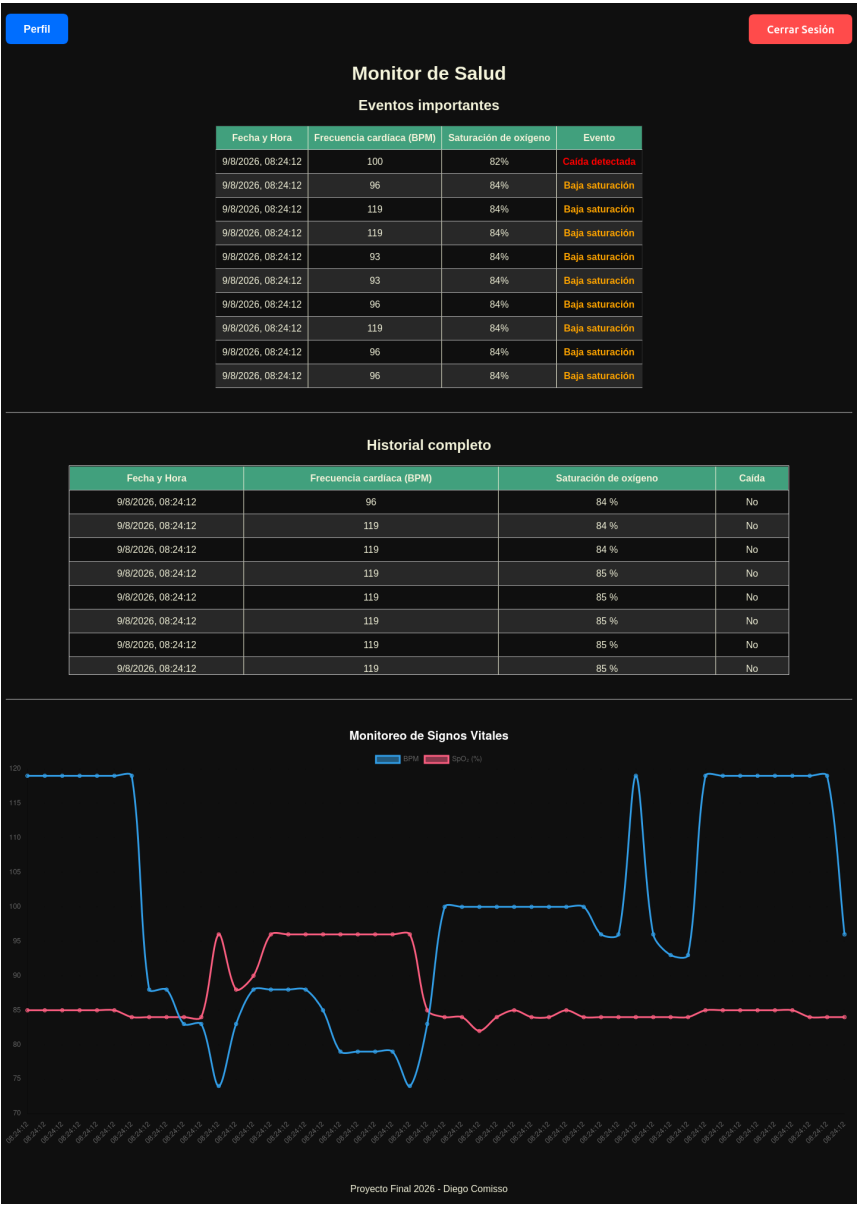
La plataforma web constituye la interfaz de interacción entre el sistema y los usuarios.

Desde ella es posible consultar el historial de mediciones, visualizar gráficos con la evolución de los signos vitales y administrar la información correspondiente a usuarios y dispositivos.

La aplicación obtiene los datos realizando solicitudes HTTP a la API REST, la cual responde con la información almacenada en formato JSON para ser procesada y presentada de forma clara al usuario.

Figura 9

Página de visualización de datos obtenidos.



Diseño del hardware

El diseño del hardware constituye una de las etapas fundamentales del proyecto, ya que define la estructura física del sistema encargado de adquirir la información biométrica del usuario. Para ello se seleccionaron componentes electrónicos que ofrecieran un equilibrio entre costo, facilidad de integración, disponibilidad en el mercado y prestaciones técnicas adecuadas para el desarrollo de un sistema de monitoreo remoto basado en Internet de las Cosas (IoT).

El dispositivo fue diseñado utilizando un microcontrolador ESP32 como unidad central de procesamiento, encargado de controlar el funcionamiento de los sensores, procesar la información obtenida y gestionar la comunicación con el servidor mediante una conexión Wi-Fi.

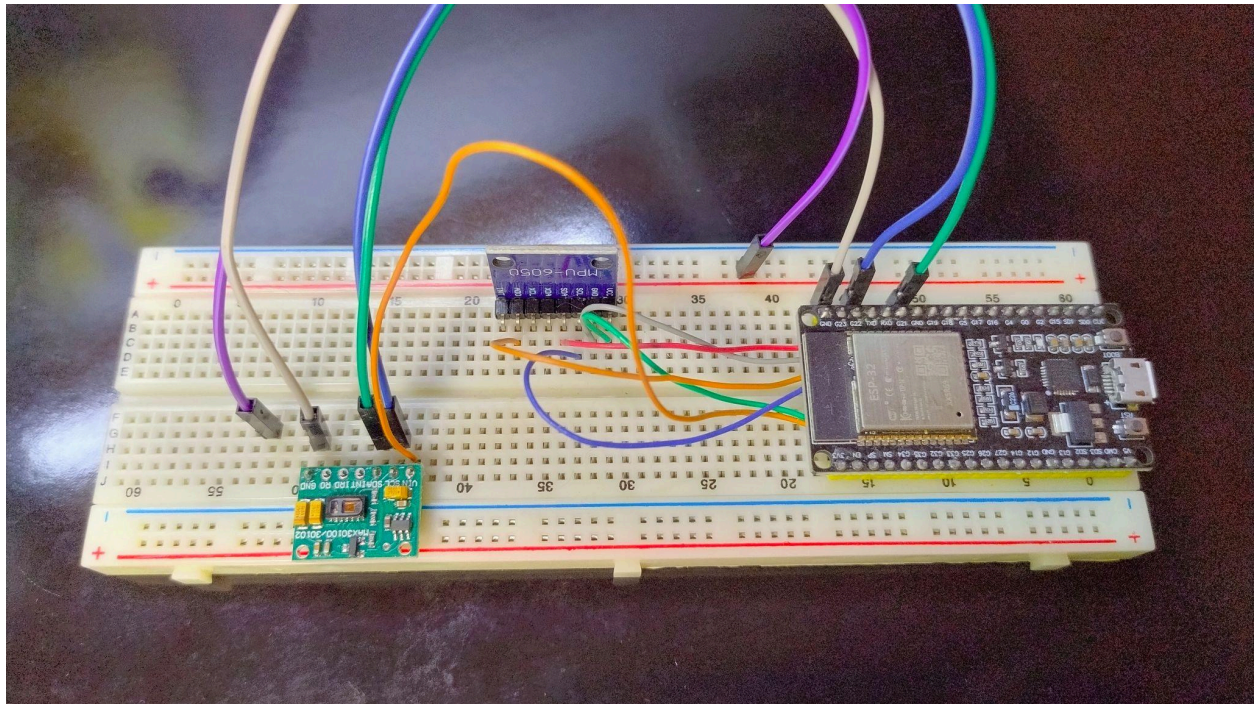
Para la adquisición de los signos vitales se incorporó un sensor biométrico MAX30102, responsable de medir la frecuencia cardíaca y la saturación de oxígeno en sangre. Asimismo, se integró un sensor MPU6050, utilizado para registrar los movimientos del dispositivo y detectar posibles caídas mediante el análisis de la aceleración.

Todos los componentes fueron montados inicialmente sobre una placa de pruebas (protoboard), permitiendo realizar modificaciones durante la etapa de desarrollo y validar el funcionamiento de cada uno de los módulos antes de su integración definitiva.

El diseño del hardware fue concebido con una arquitectura modular, permitiendo reemplazar o incorporar nuevos sensores sin modificar significativamente el resto del sistema. Esta característica facilita futuras ampliaciones y contribuye a la escalabilidad de la solución desarrollada.

Figura 10

Componentes principales del hardware.



Nota: Fotografía general del dispositivo mostrando el ESP32, el sensor MAX30102 y el sensor MPU6050.

Tabla 7*Componentes utilizados en el hardware.*

Componentes	Función
ESP32	Procesamiento y comunicación Wi-Fi
MAX30102	Medición de frecuencia cardíaca y SpO ₂
MPU6050	Detección de movimiento y caídas
Protoboard	Prototipado del circuito
Cables Dupont	Interconexión de los componentes

Tabla 8*Conexiones entre los componentes.*

Componentes	Pin del módulo	Pin ESP32	Función
MAX30102	SDA	GPIO 21	Bus I ² C
MAX30102	SCL	GPIO 22	Bus I ² C
MAX30102	VCC	3.3 V	Alimentación
MAX30102	GND	GND	Tierra
MPU6050	SDA	GPIO 21	Bus I ² C
MPU6050	SCL	GPIO 22	Bus I ² C
MPU6050	VCC	3.3 V	Alimentación
MPU6050	GND	GND	Tierra

Inicialización del sistema

Al energizar el dispositivo, el ESP32 comienza la ejecución del firmware iniciando una serie de tareas destinadas a preparar todos los componentes necesarios para el funcionamiento del sistema de monitoreo.

La primera etapa consiste en la configuración de los recursos internos del microcontrolador y la inicialización del puerto serie, utilizado durante el desarrollo para visualizar mensajes de depuración y verificar el correcto funcionamiento del dispositivo mediante el Monitor Serie del entorno Arduino IDE.

Posteriormente se inicializa el bus de comunicación I²C, encargado de establecer el intercambio de información entre el ESP32 y los sensores MAX30102 y MPU6050. Una vez habilitado este canal de comunicación, el firmware verifica la disponibilidad de ambos dispositivos antes de continuar con la ejecución del programa.

Luego se realiza la configuración del sensor biométrico MAX30102, estableciendo los parámetros necesarios para la adquisición de la frecuencia cardíaca y la saturación de oxígeno en sangre. De forma similar, se inicializa el sensor MPU6050 para comenzar la lectura de los valores de aceleración y velocidad angular.

Finalizada la inicialización de los sensores, el ESP32 intenta establecer la conexión con la red Wi-Fi utilizando el nombre de la red (SSID) y la contraseña previamente configurados en el firmware. Mientras la conexión no se establece correctamente, el dispositivo permanece realizando nuevos intentos de conexión.

Una vez obtenida una dirección IP asignada por la red, el sistema informa mediante el Monitor Serie que la conexión fue realizada correctamente y queda preparado para comenzar el ciclo de adquisición de datos.

Completadas todas estas etapas, el firmware ingresa al ciclo principal de ejecución, donde realiza de forma repetitiva la lectura de los sensores, el procesamiento de la información y el envío de las mediciones hacia el servidor.

Esta secuencia de inicialización garantiza que todos los componentes del sistema se encuentren operativos antes de comenzar el monitoreo, reduciendo la posibilidad de errores durante el funcionamiento del dispositivo.

Figura 11

Proceso de inicialización del sistema.

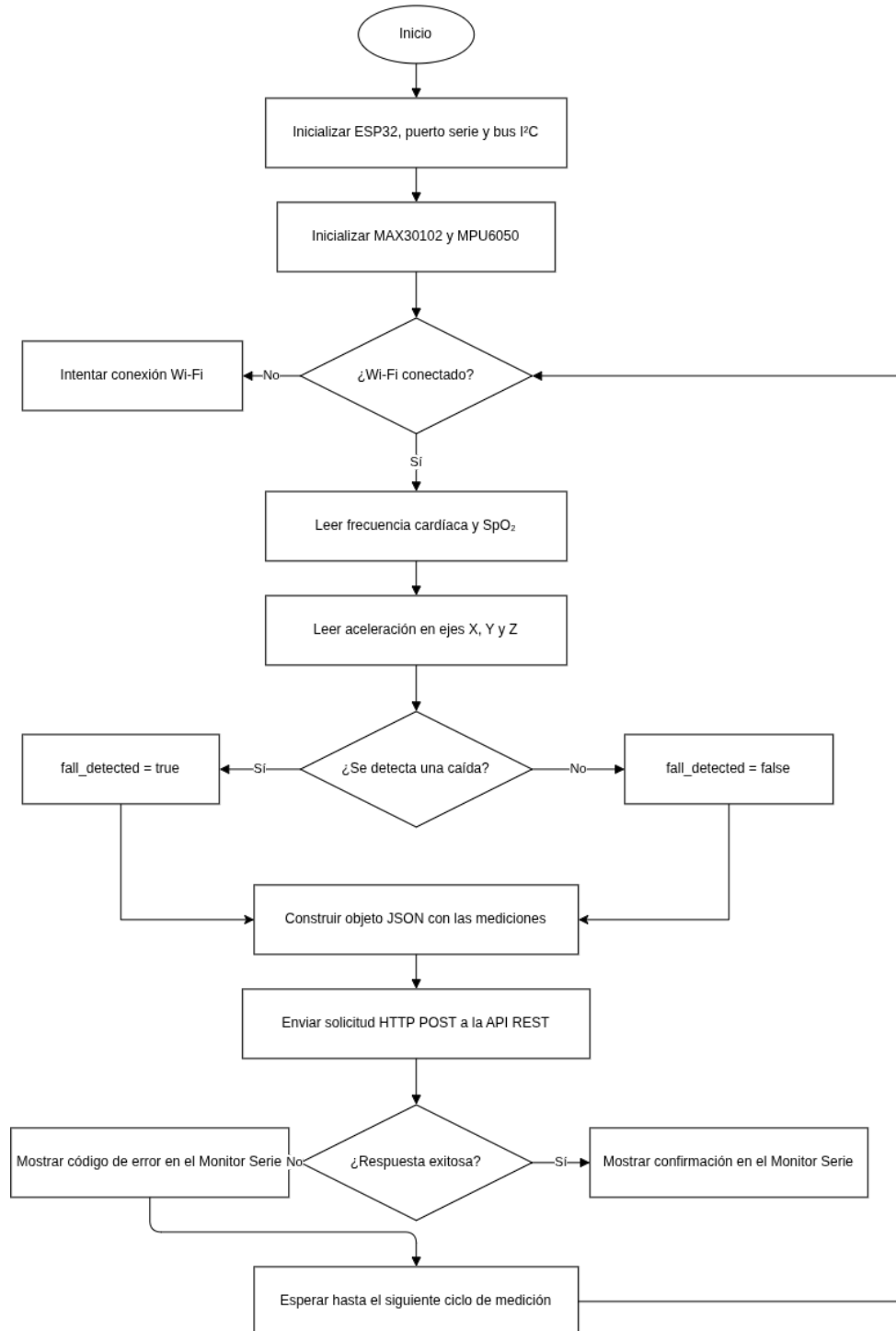
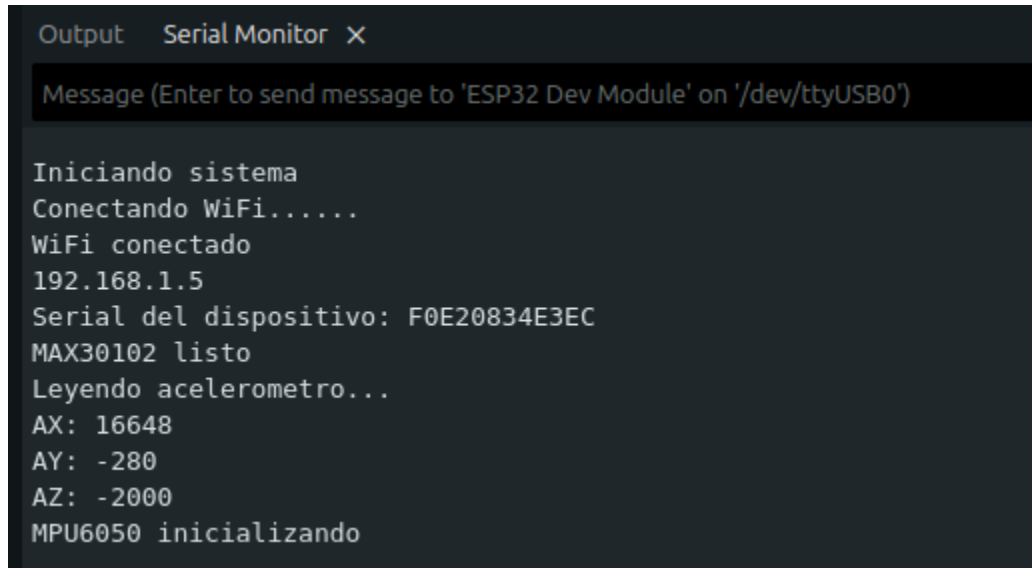


Figura 12

Monitor Serie mostrando la inicialización del sistema.



```
Output  Serial Monitor  X
Message (Enter to send message to 'ESP32 Dev Module' on '/dev/ttyUSB0')

Iniciando sistema
Conectando WiFi.....
WiFi conectado
192.168.1.5
Serial del dispositivo: F0E20834E3EC
MAX30102 listo
Leyendo acelerometro...
AX: 16648
AY: -280
AZ: -2000
MPU6050 inicializando
```

Detección de caídas

La detección de caídas fue implementada utilizando el sensor MPU6050, el cual proporciona información sobre la aceleración y la velocidad angular del dispositivo en los tres ejes espaciales. Durante el funcionamiento normal del sistema, el ESP32 realiza lecturas periódicas de estos valores y analiza continuamente el comportamiento del movimiento.

El algoritmo desarrollado se basa en la comparación de los valores de aceleración obtenidos con umbrales previamente establecidos durante la etapa de pruebas.

Cuando el sistema detecta una variación brusca de aceleración compatible con una posible caída, el evento es identificado y registrado por el firmware.

El registro de estos eventos resulta de gran utilidad, ya que permite consultar posteriormente el momento en que ocurrió la caída y relacionarlo con las mediciones biométricas obtenidas durante ese mismo período.

Es importante destacar que el algoritmo implementado constituye una aproximación al problema de la detección de caídas y fue desarrollado con fines académicos. La confiabilidad del sistema puede incrementarse en futuras versiones mediante la incorporación de técnicas de procesamiento digital de señales, filtros de suavizado o algoritmos basados en aprendizaje automático, capaces de mejorar la precisión de la detección.

A pesar de estas posibles mejoras, la solución desarrollada demostró un funcionamiento adecuado para los objetivos planteados en el Trabajo Final Integrador, integrando el monitoreo de signos vitales con la detección de eventos de riesgo dentro de una única plataforma.

Figura 13

Líneas de código de la función de detección de caídas.

```
// =====  
// FUNCION: DETECTAR CAIDA  
// =====  
  
bool detectarCaida()  
{  
    // Convertir lecturas del giroscopio a Grados por Segundo (°/s)  
    float gyroX = gx / 131.0;  
    float gyroY = gy / 131.0;  
    float gyroZ = gz / 131.0;  
  
    // Calcular la magnitud total del giro  
    float rotMagnitude = sqrt(  
        gyroX * gyroX +  
        gyroY * gyroY +  
        gyroZ * gyroZ  
    );  
  
    // Si gira a más de 200 grados por segundo, se considera un volteo brusco  
    if (rotMagnitude > 200.0)  
    {  
        return true;  
    }  
  
    return false;  
}
```

Desarrollo del Backend

Su desarrollo se realizó utilizando Node.js como entorno de ejecución y Express como framework para la implementación de la API REST. Esta combinación permitió construir una aplicación modular, escalable y capaz de gestionar simultáneamente las solicitudes provenientes tanto del dispositivo de monitoreo como de los usuarios de la plataforma.

La arquitectura implementada sigue el modelo Cliente-Servidor, donde el ESP32 y la aplicación web actúan como clientes que realizan solicitudes HTTP al servidor. El backend procesa cada solicitud, ejecuta la lógica correspondiente y responde con la información solicitada o con el resultado de la operación realizada.

Organización del proyecto (Arquitectura basada en MVC)

Con el objetivo de desarrollar una aplicación organizada, escalable y de fácil mantenimiento, el backend fue estructurado siguiendo una arquitectura modular basada en el patrón Modelo-Vista-Controlador (MVC). Este patrón propone separar las distintas responsabilidades de la aplicación en componentes independientes, favoreciendo la reutilización del código y simplificando las tareas de desarrollo y mantenimiento.

La estructura implementada divide la aplicación en diferentes módulos, cada uno encargado de una función específica dentro del sistema.

Los Modelos (Models) representan la estructura de los datos almacenados en la base de datos y permiten realizar operaciones de consulta, inserción, actualización y eliminación de registros. En el presente proyecto, los modelos fueron desarrollados utilizando Sequelize como ORM, estableciendo las relaciones entre usuarios, registros biométricos, enfermedades y cuidadores.

Los Controladores (Controllers) contienen la lógica de negocio de la aplicación. Son responsables de recibir las solicitudes provenientes de las rutas, validar la información

recibida, ejecutar las operaciones correspondientes sobre la base de datos y generar la respuesta que será enviada al cliente.

Las Rutas (Routes) constituyen el punto de entrada de cada solicitud HTTP. Su función consiste en asociar cada endpoint de la API REST con el controlador correspondiente, manteniendo organizada la estructura de la aplicación y facilitando la incorporación de nuevas funcionalidades.

Además de estos componentes principales, el proyecto incorpora otros módulos complementarios que contribuyen a mejorar la organización del software.

La carpeta Middlewares contiene funciones que se ejecutan antes de llegar a los controladores, permitiendo realizar tareas como autenticación, autorización, validación de información y control de acceso a determinados recursos del sistema.

La carpeta Config reúne la configuración general de la aplicación, incluyendo los parámetros de conexión con la base de datos y otras variables necesarias para el funcionamiento del servidor.

La estructura modular implementada permitió separar claramente las responsabilidades de cada componente, reduciendo el acoplamiento entre las diferentes partes del sistema y facilitando futuras tareas de mantenimiento o ampliación.

Figura 14

Estructura de carpetas del proyecto.

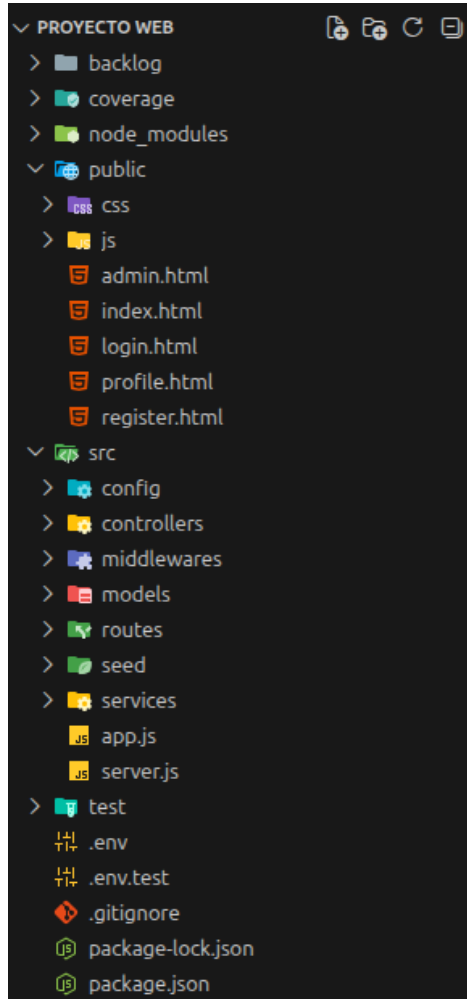


Tabla 9

Resumen de la función de los principales directorios utilizados durante el desarrollo del backend.

Directorio	Función
config	Configuración del servidor y conexión con la base de datos.
controllers	Implementación de la lógica de negocio de la aplicación.
middlewares	Validación de solicitudes y control de acceso.
models	Definición de las entidades y relaciones de la base de datos mediante Sequelize.
routes	Definición de los endpoints de la API REST.
public	Archivos estáticos utilizados por la plataforma web.
app.js	Punto de entrada y configuración principal del servidor.

Gestión de usuarios y autenticación

Uno de los principales módulos desarrollados para la plataforma corresponde a la administración de usuarios y al sistema de autenticación. Este componente permite controlar el acceso a la aplicación, gestionar la información de los distintos usuarios registrados y restringir determinadas funcionalidades según el rol asignado a cada uno.

El sistema fue diseñado para contemplar diferentes perfiles de usuario, cada uno con permisos específicos de acuerdo con sus responsabilidades dentro de la plataforma.

Los roles implementados son:

Administrador: posee acceso completo a todas las funcionalidades del sistema, incluyendo la gestión de usuarios, dispositivos, enfermedades y registros biométricos.

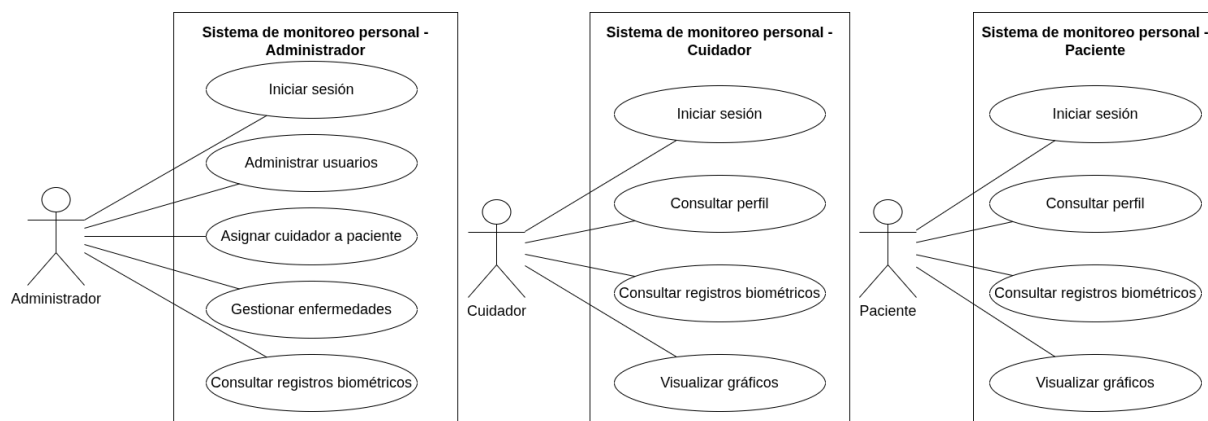
Paciente: corresponde a la persona que utiliza el dispositivo de monitoreo. Su información personal y sus registros de salud son almacenados en la plataforma para su consulta.

Cuidador: usuario autorizado para realizar el seguimiento de uno o más pacientes, permitiendo consultar las mediciones registradas y la información asociada a las personas bajo su cuidado.

Esta organización permite mantener un adecuado control sobre el acceso a la información, garantizando que cada usuario únicamente pueda acceder a las funcionalidades correspondientes a su perfil.

Figura 15

Diagramas de caso de uso del sistema.



Integración del backend con la plataforma web

La interacción entre ambos componentes se realiza mediante solicitudes HTTP enviadas desde la aplicación web hacia los distintos endpoints implementados en el servidor. Cada solicitud es procesada por la API REST, la cual ejecuta la lógica correspondiente y devuelve una respuesta en formato JSON con la información solicitada.

Gracias a este mecanismo, la plataforma puede mostrar el historial de mediciones, administrar usuarios, consultar la información de pacientes y cuidadores, así como visualizar los eventos de caída registrados por el dispositivo.

La utilización de una API REST permitió desacoplar completamente el frontend del backend. Esta separación facilita el mantenimiento del sistema y permite desarrollar nuevas interfaces, como aplicaciones móviles o clientes de escritorio, sin necesidad de modificar la lógica implementada en el servidor.

Asimismo, la arquitectura desarrollada posibilita que múltiples clientes puedan acceder simultáneamente a la información almacenada, manteniendo una única fuente de datos centralizada y garantizando la consistencia de la información.

Desarrollo de la Plataforma Web

La plataforma web fue desarrollada con el objetivo de proporcionar una interfaz sencilla e intuitiva para la administración del sistema de monitoreo y la consulta de la información generada por el dispositivo basado en ESP32.

A través de esta aplicación, los distintos usuarios pueden acceder a las funcionalidades correspondientes a su perfil, consultar las mediciones almacenadas, administrar la información de pacientes y cuidadores y visualizar el historial de registros biométricos.

El desarrollo de la interfaz se realizó utilizando tecnologías web ampliamente utilizadas, entre ellas HTML, CSS y JavaScript para el frontend, mientras que la comunicación con el servidor se implementó mediante solicitudes HTTP dirigidas a la API REST desarrollada con Node.js y Express.

Inicio de sesión

El acceso a la plataforma web se realiza mediante un sistema de autenticación basado en credenciales, cuyo objetivo es garantizar que únicamente los usuarios registrados puedan acceder a la información y funcionalidades disponibles.

Al ingresar a la aplicación, el usuario visualiza una pantalla de inicio de sesión donde debe introducir su dirección de correo electrónico y contraseña. Estos datos son

enviados mediante una solicitud HTTP de tipo POST hacia la API REST, donde son validados antes de permitir el acceso al sistema.

El servidor verifica que las credenciales ingresadas correspondan a un usuario registrado en la base de datos. En caso de que la autenticación sea exitosa, el usuario obtiene acceso a la plataforma con los permisos asociados a su rol. Si las credenciales son incorrectas o el usuario no existe, el sistema informa el error mediante un mensaje que solicita verificar la información ingresada.

La autenticación constituye una medida de seguridad fundamental, ya que protege la información almacenada y restringe el acceso únicamente a personas autorizadas.

Una vez iniciada la sesión, la plataforma identifica el perfil del usuario autenticado y habilita únicamente las funcionalidades correspondientes a dicho rol.

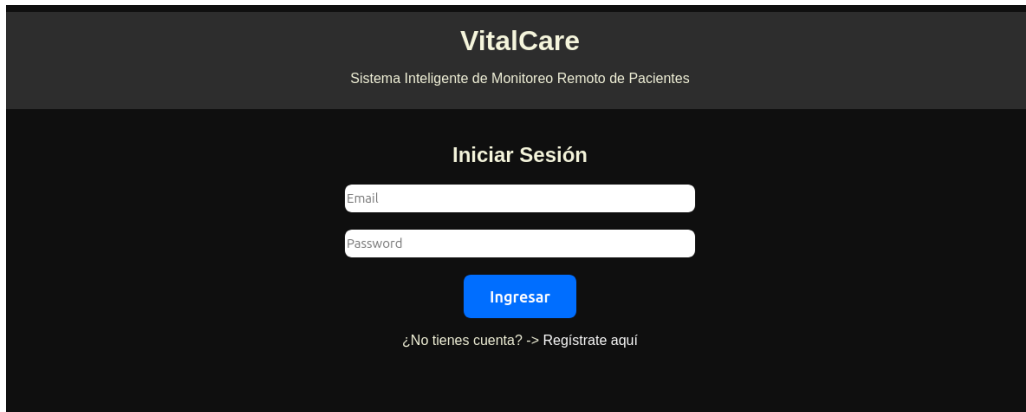
- Los administradores poseen acceso completo a la administración del sistema.
- Los pacientes pueden consultar su información personal y sus registros biométricos.
- Los cuidadores pueden acceder a la información de los pacientes que tienen asignados y consultar la evolución de sus mediciones

Esta diferenciación de permisos permite mantener la confidencialidad de la información y evita que usuarios sin autorización accedan a datos que no les corresponden.

Durante el desarrollo se realizaron pruebas de autenticación utilizando distintos perfiles de usuario para verificar el correcto funcionamiento del sistema de acceso, la validación de credenciales y la asignación de permisos.

Figura 16

Pantalla inicio de sesión.



VitalCare
Sistema Inteligente de Monitoreo Remoto de Pacientes

Iniciar Sesión

Email

Password

Ingresar

¿No tienes cuenta? -> Regístrate aquí

Registro de usuarios

El sistema permite registrar nuevos usuarios mediante un formulario donde se ingresan los datos personales requeridos para su identificación.

Entre la información solicitada se encuentran:

- Nombre y apellido.
- Documento Nacional de Identidad (DNI).
- Fecha de nacimiento.
- Dirección de correo electrónico.
- Contraseña.
- Rol asignado dentro del sistema.

Cuando el usuario registrado corresponde al rol de cuidador, el formulario habilita además campos específicos para ingresar el número telefónico y seleccionar el paciente que tendrá asignado. Por el contrario, cuando se registra un paciente, es posible asociar las enfermedades correspondientes mediante la selección de las opciones disponibles.

Figura 17

Pantalla de registro de usuario paciente.

VitalCare

Sistema Inteligente de Monitoreo Remoto de Pacientes

Crear cuenta

Fecha de nacimiento:

Registrarse como:

Usuario / Paciente

¿Presenta alguna de estas enfermedades?

☐

Artrosis y Artritis
Afecciones crónicas que causan dolor, inflamación y pérdida de movilidad en las articulaciones.

☐

Depresión y aislamiento
Problemas de salud mental y anímica, a menudo desencadenados por pérdidas o cambios en el estilo de vida.

☐

Diabetes (Tipo 2)
Alteración en los niveles de glucosa en sangre. Requiere control constante para evitar daño renal o visual.

☐

Enfermedad de Alzheimer y otras demencias
Trastornos neurodegenerativos que afectan la memoria, el pensamiento y la autonomía.

☐

Enfermedad Pulmonar Obstructiva Crónica (EPOC)
Afección inflamatoria pulmonar que causa dificultad para respirar, frecuentemente asociada al tabaquismo.

☐

Enfermedades cardiovasculares
Incluyen insuficiencia cardíaca, arritmias y antecedentes de infartos o accidentes cerebrovasculares (ictus).

☐

Hipertensión arterial
Aumento crónico de la presión sanguínea en los vasos sanguíneos, principal factor de riesgo para problemas cardíacos.

☐

Osteoporosis
Disminución de la densidad ósea que hace que los huesos sean más frágiles y propensos a fracturas.

☐

Problemas visuales y auditivos
Pérdida gradual de la audición (presbiacusia) y enfermedades oculares como cataratas o errores de refracción.

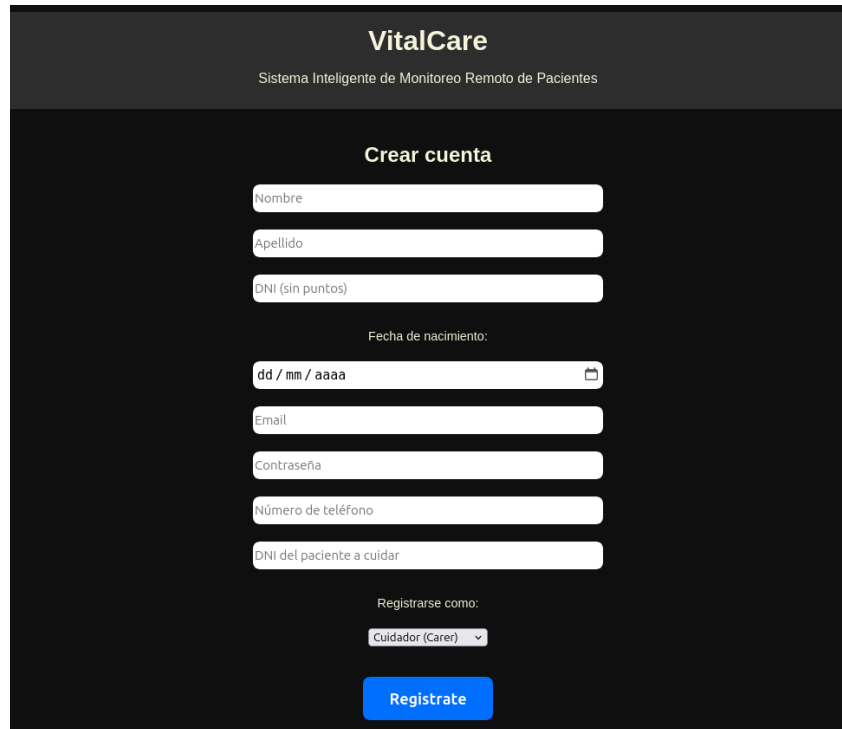
☐

Síndromes geriátricos
Incluyen problemas de incontinencia urinaria, caídas frecuentes o riesgo de fragilidad y desnutrición.

Registrarse

Figura 18

Pantalla de registro de usuario cuidador.



The screenshot shows a web interface for 'VitalCare', described as 'Sistema Inteligente de Monitoreo Remoto de Pacientes'. The main heading is 'Crear cuenta'. Below this, there are several input fields: 'Nombre', 'Apellido', 'DNI (sin puntos)', 'Fecha de nacimiento:' (with a date picker showing 'dd / mm / aaaa'), 'Email', 'Contraseña', 'Número de teléfono', and 'DNI del paciente a cuidar'. At the bottom, there is a section 'Registrarse como:' with a dropdown menu currently showing 'Cuidador (Carer)' and a blue 'Registrate' button.

Monitoreo de pacientes y visualización de registros biométricos

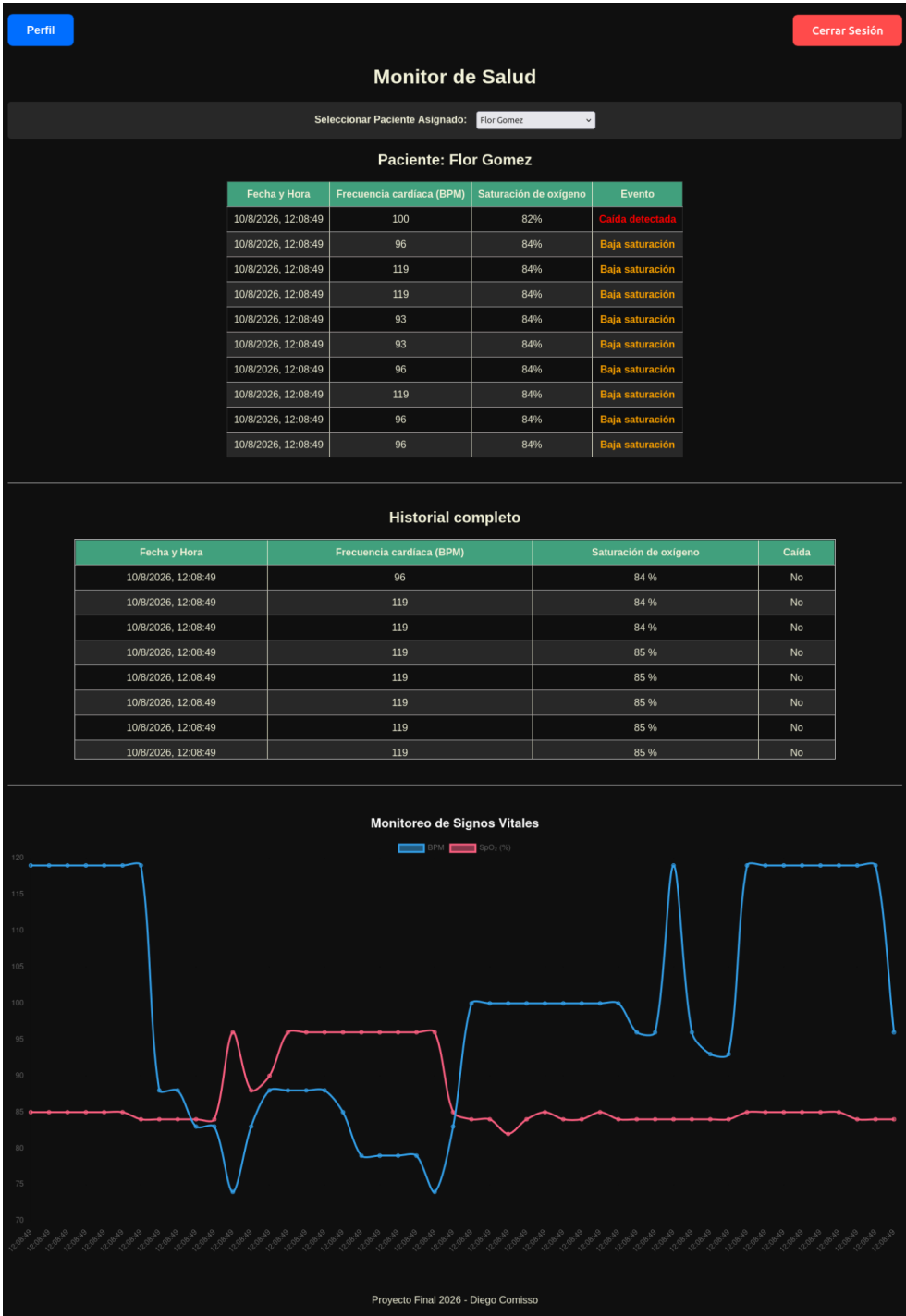
La principal funcionalidad de la plataforma consiste en permitir el monitoreo remoto del estado de los pacientes mediante la consulta de las mediciones enviadas por el dispositivo basado en ESP32. Para ello, la aplicación web proporciona una interfaz que permite acceder al historial de registros biométricos almacenados en la base de datos y visualizar la información de manera clara y organizada.

Cada vez que el dispositivo realiza una nueva medición, los datos son transmitidos al servidor mediante la API REST y almacenados en la base de datos. Posteriormente,

estos registros pueden ser consultados desde la plataforma web por los usuarios autorizados, permitiendo realizar un seguimiento continuo de la evolución del paciente.

Figura 19

Pantalla monitoreo del usuario cuidador.



Beneficios del monitoreo remoto

La integración entre el dispositivo de adquisición de datos y la plataforma web permite realizar un seguimiento continuo del estado del paciente sin necesidad de acceder físicamente al dispositivo.

La disponibilidad del historial de mediciones facilita la consulta de información pasada, el análisis de la evolución de los signos vitales y la identificación de posibles eventos relevantes registrados durante el funcionamiento del sistema.

Esta arquitectura proporciona una herramienta de apoyo para el monitoreo remoto, permitiendo centralizar toda la información generada por el dispositivo en una única plataforma accesible para los usuarios autorizados.

Resultados obtenidos

El desarrollo del presente Trabajo Final Integrador permitió implementar un sistema funcional de monitoreo remoto de signos vitales basado en tecnologías IoT. La solución integra hardware, firmware y software en una única plataforma capaz de adquirir, transmitir, almacenar y visualizar información biométrica de manera organizada.

Durante las pruebas realizadas se verificó el correcto funcionamiento de cada uno de los componentes desarrollados, comprobando que el dispositivo basado en ESP32 logra establecer comunicación con los sensores, procesar las mediciones obtenidas y transmitir la información hacia el servidor utilizando una conexión Wi-Fi.

Asimismo, la API REST desarrollada con Node.js y Express recibió correctamente las solicitudes provenientes del dispositivo, validó la información recibida y almacenó cada medición en la base de datos, permitiendo posteriormente su consulta desde la plataforma web.

La aplicación desarrollada posibilitó la administración de usuarios, pacientes y cuidadores, así como la consulta del historial de registros biométricos y la visualización gráfica de la evolución de los signos vitales registrados por el sistema.

La implementación del algoritmo de detección de caídas permitió registrar eventos de movimiento compatibles con posibles caídas, incorporando esta información al historial de mediciones del paciente y ampliando las capacidades del sistema de monitoreo.

En términos generales, los resultados obtenidos permitieron verificar el cumplimiento de los objetivos planteados al inicio del proyecto. Se logró desarrollar una solución capaz de integrar la adquisición de datos biométricos, la comunicación inalámbrica, el almacenamiento de información y la visualización remota dentro de una arquitectura organizada y escalable.

Si bien el sistema alcanzó los objetivos propuestos, durante el desarrollo se identificaron algunas posibilidades de mejora que podrán ser abordadas en trabajos futuros. Entre ellas se destacan la incorporación de un módulo GPS para la geolocalización del usuario, la implementación de notificaciones automáticas ante eventos críticos, la integración con aplicaciones móviles y la utilización de algoritmos más avanzados para mejorar la precisión de la detección de caídas.

Los resultados obtenidos demuestran la viabilidad de utilizar tecnologías IoT y plataformas web para el desarrollo de sistemas de monitoreo remoto orientados al cuidado de personas, constituyendo una base sólida para futuras ampliaciones y aplicaciones en el ámbito de la salud.

Conclusión

El presente Trabajo Final Integrador tuvo como objetivo el desarrollo de un sistema de monitoreo remoto de signos vitales basado en tecnologías IoT, integrando un dispositivo electrónico para la adquisición de datos biométricos con una plataforma web capaz de almacenar, procesar y visualizar la información obtenida.

A lo largo del desarrollo fue posible integrar componentes de hardware y software dentro de una arquitectura Cliente-Servidor, permitiendo la comunicación entre el dispositivo basado en ESP32, la API REST, la base de datos y la plataforma web. La implementación de esta arquitectura permitió comprobar la viabilidad de utilizar tecnologías de bajo costo para desarrollar soluciones orientadas al monitoreo remoto de personas.

Aportes del proyecto

El sistema desarrollado demuestra que es posible integrar tecnologías IoT con aplicaciones web para implementar soluciones orientadas al monitoreo remoto de personas.

Entre los principales aportes del proyecto pueden destacarse la integración de múltiples sensores biométricos dentro de un único dispositivo, la implementación de una arquitectura modular basada en una API REST y el desarrollo de una plataforma web que permite administrar usuarios y consultar el historial de mediciones desde cualquier equipo conectado a la red.

Otro aspecto relevante del trabajo consiste en la incorporación de un mecanismo de detección de caídas, complementando la información fisiológica obtenida por el sensor biométrico y proporcionando una herramienta adicional para el seguimiento del estado del paciente.

La arquitectura implementada facilita además la incorporación de nuevos dispositivos y funcionalidades sin requerir modificaciones significativas en la estructura general del sistema, constituyendo una base sólida para futuros desarrollos.

Limitaciones y trabajos futuros

Si bien el sistema desarrollado cumplió los objetivos establecidos, durante su implementación se identificaron diferentes aspectos susceptibles de mejora.

Una de las principales limitaciones corresponde a la detección de caídas, la cual se basa en un algoritmo de comparación de umbrales. Si bien los resultados obtenidos fueron satisfactorios durante las pruebas realizadas, podrían implementarse algoritmos más avanzados que incrementen la precisión y reduzcan la probabilidad de falsas detecciones.

Asimismo, el proyecto no incorpora un sistema de geolocalización que permita conocer la ubicación del usuario al producirse una emergencia. La incorporación de un módulo GPS constituiría una mejora importante para futuras versiones del dispositivo.

Entre las posibles líneas de trabajo futuro también pueden mencionarse:

- Desarrollo de una aplicación móvil para Android e iOS.
- Incorporación de notificaciones automáticas ante eventos críticos.
- Incorporación de nuevos sensores biométricos para ampliar las capacidades de monitoreo.
- Optimización del consumo energético para incrementar la autonomía del dispositivo.

Consideraciones finales

El desarrollo del presente Trabajo Final Integrador permitió aplicar de forma integrada los conocimientos adquiridos durante la carrera en áreas como electrónica,

programación de sistemas embebidos, desarrollo web, bases de datos, redes de comunicación y arquitectura de software.

La experiencia obtenida durante la realización del proyecto permitió comprender las distintas etapas involucradas en el desarrollo de una solución tecnológica completa, desde el diseño del hardware hasta la implementación del software y la integración de todos sus componentes.

Finalmente, el sistema desarrollado constituye un prototipo funcional que demuestra la viabilidad de implementar soluciones de monitoreo remoto utilizando tecnologías accesibles y de bajo costo. Si bien existen posibilidades de mejora, los resultados obtenidos permiten considerar que el proyecto representa una base sólida para futuras investigaciones y desarrollos orientados al cuidado y seguimiento de personas mediante dispositivos IoT.

Referencias

Analog Devices. (s. f.). *MAX30102: High-sensitivity pulse oximeter and heart-rate sensor for wearable health*. Analog Devices.

<https://www.analog.com/en/products/max30102.html>

Apple. (s. f.). *Cómo usar la app Oxígeno en Sangre en el Apple Watch*. Soporte técnico de Apple. <https://support.apple.com/es-la/120358>

Apple. (s. f.). *Control de la frecuencia cardiaca con el Apple Watch*. Soporte técnico de Apple. <https://support.apple.com/es-la/120277>

Apple. (s. f.). *Usar Detección de Caídas con el Apple Watch*. Soporte técnico de Apple. <https://support.apple.com/es-la/108896>

Chart.js. (s. f.). *Chart.js documentation*. <https://www.chartjs.org/docs/latest/>

Espressif Systems. (s. f.). *ESP-IDF Programming Guide: ESP32*.

<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/>

Express.js. (s. f.). *Express: Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com/>

Garmin. (s. f.). *Pulsioximetría: Tecnología Garmin*.

<https://www.garmin.com/es-ES/garmin-technology/health-science/pulse-ox/>

Garmin. (s. f.). *Incident detection and assistance requirements*. Garmin Support.

<https://support.garmin.com/>

MySQL. (2026). *MySQL 8.4 Reference Manual*. Oracle.

<https://dev.mysql.com/doc/refman/8.4/en/>

Node.js. (s. f.). *Node.js documentation*. <https://nodejs.org/docs/latest/api/>

Samsung Electronics. (s. f.). *Galaxy Watch*. Samsung.

<https://www.samsung.com/ar/watches/>

Sequelize. (2026). *Sequelize v6 documentation*. <https://sequelize.org/docs/v6/>

TDK InvenSense. (s. f.). *MPU-6050 six-axis MEMS motion tracking device*. TDK

InvenSense.

<https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/>

Anexos

1. Código fuente del firmware del ESP32

<https://github.com/diegoec55/ProyectoFinal2026-Hardware>

2. Código fuente del backend

<https://github.com/diegoec55/ProyectoFinal2026-Software>

3. Modelo de la base de datos

